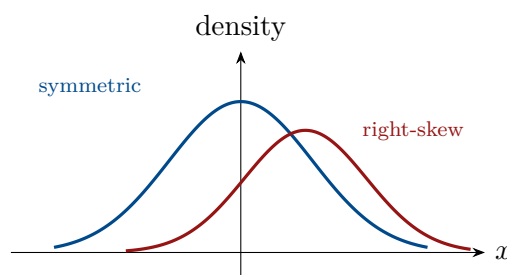

Exploratory Data Analysis

A Book for Absolute Beginners

From dirty tables → measurement scales → association, derived step by step

No prior stats assumed — every formula on the class board is unpacked here



Prepared by

Krushna Parmar

Roll No. **202611032** • Course IT462

Grounded in the IT462 lecture boards (23 Jul – 20 Aug 2026) and the surrounding theory those boards assume.

Every idea is explained in plain words first, then derived, then drawn, then written as Python — with Gujarat-style analogies, xkcd, and the 2026 toolkit (Polars, DuckDB, honest LLM-EDA).

Contents

How to Read This Book	iii
1 Data, Information, and the Three Analyses	1
1.1 The only two tools that actually matter	1
1.2 Data is the pile. Information is the sentence.	2
1.3 Structured vs unstructured	4
1.4 Where the data came from: primary, secondary, tertiary	4
1.5 File formats you will actually open	5
1.6 IDA: making the pile usable	6
1.7 The dirty table from the board	7
1.8 IDA, then EDA, then CDA	9
1.9 The anatomy of a structured table	9
2 Measurement Scales: NOIR	11
2.1 The fork, then the ladder	11
2.2 The property table (memorise this)	12
2.3 Nominal, slowly	12
2.4 Ordinal, and the Likert scale	13
2.5 The board's transform: ordinal $\rightarrow$ interval via Likert	14
2.6 Interval vs ratio, and why age is ratio	15
2.7 Which operations are legal?	16
2.8 The decision tree from Lecture 6	16
3 Central Tendency and Dispersion	19
3.1 The centre: three pins, not one	19
3.1.1 Arithmetic mean	19
3.1.2 Geometric and harmonic means	20
3.1.3 Median	21
3.1.4 Mode	22
3.2 Dispersion: how wide is the pile?	22
3.2.1 Range	23
3.2.2 Variance: the average squared red length	23
3.2.3 Standard deviation	24
4 Percentiles, Shape, and Association	27
4.1 Relative standing	27
4.1.1 The ten numbers from the board	28
4.2 Skewness: the pile's lean	29
4.3 Kurtosis: tailedness / peakedness	30
4.4 Association: how X and Y move	32
4.4.1 Covariance, then Pearson	32

4.4.2	Spearman, point-biserial, Cramér	33
5	Populations, Samples, and Experiments	37
5.1	From a column to a world	37
5.2	Parameter vs statistic	37
5.3	Three methods from the board (and the cousins)	38
5.4	Experimental design	40
6	After Import: The Practitioner's Loop	43
6.1	The feature checklist	43
6.2	The value checklist	44
6.3	Encoding: how NOIR enters a model	45
6.4	A visual grammar (so your plots are not souvenirs)	46
6.5	The 2026 stack, without throwing away the board	47
6.6	A full walk on the dirty table	49
	Closing	51
	Bibliography	53

How to Read This Book

This book assumes **you have never taken a statistics course**. It is the sister of the FOML theory book: same voice, same boxes, a different subject. EDA is not “plot stuff until it looks pretty”. It is the disciplined habit of *looking* before you *model*.

Each idea appears in several forms so at least one clicks:

- **From the class board** (orange) — the exact claim, table, or formula *sir* wrote.
- **In Plain Words** (green) — zero jargon first.
- **Derivation — step by step** (blue) — every algebra line, nothing skipped.
- **Figure** — a picture so you can *see* the maths.
- **Gujarat-Style Analogy** — farsan counters, kite season, stepwells, cricket scores. English only.
- **How to write it in Python** (grey) — the theory as `pandas/numpy/scipy` (and, later, `Polars`).
- **Watch Out** (red) — the mistakes that fail vivas and production jobs.
- **2026 cutting edge** (navy) — what people on `r/datascience` and `r/MachineLearning` actually reach for in August 2026, without abandoning the class maths.

Do not skip Chapter 1. The later chapters keep using the words *feature*, *observation*, *nominal*, *usable*. If a symbol confuses you, it was defined earlier — go back rather than guessing.

What this book is not. It is not a dump of the lecture screenshots, and it is not a pirate copy of Tukey, Wickham, or ISLP. The lecture boards are the *spine*. The surrounding chapters are the muscle you need so that after this book you are dangerous in a good way: you can classify a column on sight, pick a legal summary, refuse a fake correlation, sample without fooling yourself, and write the code.

Chapter 1

Data, Information, and the Three Analyses

1.1 The only two tools that actually matter

From the class board

IT462 — EDA. Tools: an **open mind**, a **closed heart**, Python on Google Colab, **numpy**, **pandas**.

In Plain Words

An open mind means you let the table surprise you. A closed heart means you do not fall in love with a story you brought from home — “this *must* be why sales dropped” — and then torture the columns until they confess. Colab is just a browser that already has Python. **numpy** is the fast array. **pandas** is the labelled table.

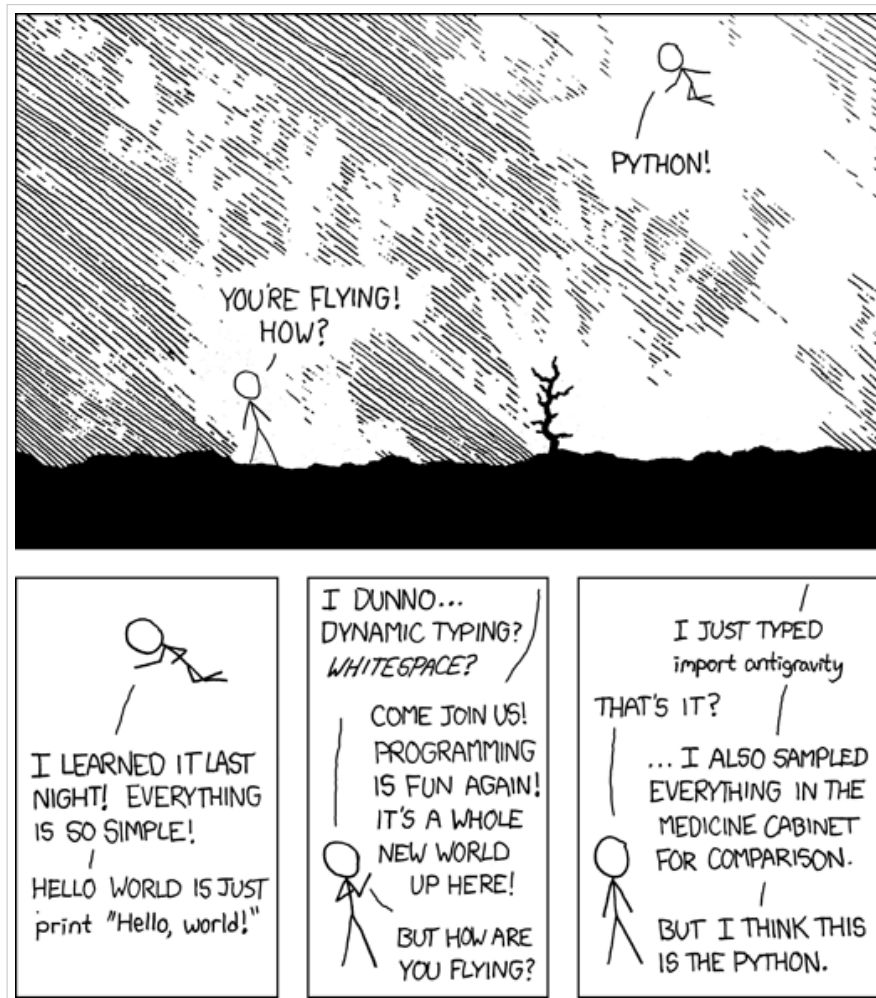
Gujarat-Style Analogy

A farsan shop at 6 pm is a dataset. An open mind tastes the sev before declaring it stale. A closed heart refuses the cousin who swears *this* batch is special because he packed it. You count, you weigh, you look. Then you talk.

How to write it in Python

```
1 import numpy as np
2 import pandas as pd
3
4 # Colab: Runtime -> Run all. Locally: pip install numpy pandas
  matplotlib scipy
5 print(np.__version__, pd.__version__)
```

The two imports at the top of every notebook this semester are not decoration. They are the tools package from the next lecture.



xkcd #353 “Python” by Randall Munroe — <https://xkcd.com/353/> (CC BY-NC 2.5).

The reason we are not doing this course in Excel macros.

1.2 Data is the pile. Information is the sentence.

From the class board

Four scores: 17, 41, 30, 99. Circled on the board: “scores with AVG” → info $\approx$ 50.

In Plain Words

Data is a collection of recorded facts. It does not yet mean anything by itself — it is four numbers on a scrap. **Information** is data after you have done something honest to it: a summary, a picture, a comparison. The board’s example takes four test scores and compresses them to an average.

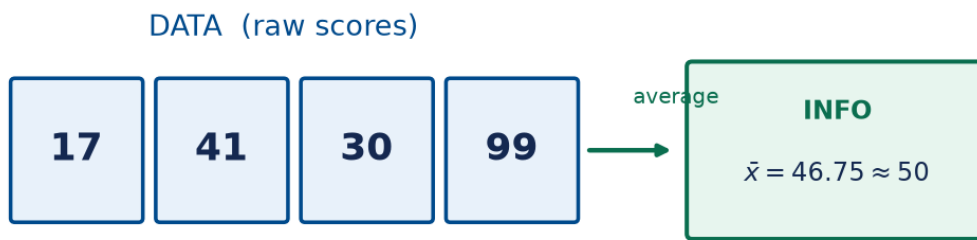


Figure — The class example. Data lives on the left. Information is the compressed number on the right.

Derivation — step by step

The arithmetic mean of n numbers $x_1, \dots, x_n$ is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Plug in the board:

$$\bar{x} = \frac{17 + 41 + 30 + 99}{4} = \frac{187}{4} = 46.75.$$

The board wrote **50**. That is a rounding-to-a-nice-number, not a different formula. In a viva, say both: “the exact mean is 46.75; on the board it was sketched as 50.” A single average is a brutal compression — 99 and 17 have been dissolved into the same soup. That is why EDA never stops at one number. (Chapter 3 will give you the whole soup menu.)

How to write it in Python

```
1 scores = np.array([17, 41, 30, 99])
2 print(scores.mean())           # 46.75
3 print(np.round(scores.mean(), -1)) # 50.0 -- what the board
  sketched
4 print(pd.Series(scores).describe()) # count, mean, std, min,
  quartiles, max
```

`describe()` is the cheapest information factory pandas owns. Use it. Then distrust it, for reasons Anscombe will explain later.

Chai-Break

If data were a cricket innings, information is the commentary sentence “they crawled to 46 for 4”. Useful. Incomplete. You still want the wagon wheel.

1.3 Structured vs unstructured

From the class board

Data: raw, unorganized / unstructured. Examples pointing at information: text, audio, visuals, records. Structured = a grid (rows $\times$ columns).

Concept

Unstructured data has no pre-agreed slots. A WhatsApp dump, a lecture recording, a folder of ID-card photos, a PDF of doctor's notes — you cannot yet ask `df['age'].mean()` because there is no **age** column. **Structured** data has been poured into a rectangle: every row is one entity, every column is one property of that entity. EDA, as taught in IT462, mostly lives on the rectangle. The modern job is often *getting* there (parsing JSON, scraping HTML, transcribing audio) and only then exploring.

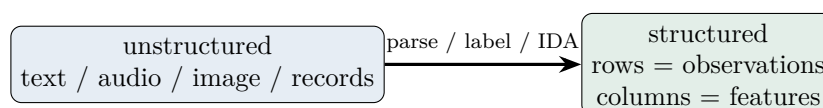


Figure — EDA starts to look like a class when the data has been forced into a grid. The forcing is already analysis.

How to write it in Python

```

1 # unstructured-ish JSON -> structured table
2 records = [
3     {"id": 2026, "IT462": 10, "age": 0, "gender": "M"},
4     {"id": 2026, "IT462": 50, "age": 10, "gender": "F"},
5 ]
6 df = pd.json_normalize(records)
7 print(df.dtypes)      # id int, IT462 int, age int, gender object (=
                        # categorical-ish)
  
```

object in pandas usually means “string / mixed / I have not thought yet”. Thinking is Chapter 2.

1.4 Where the data came from: primary, secondary, tertiary

From the class board

Source $\rightarrow$ Primary (“I am involved”) / Secondary (“you are involved”) / Tertiary.

Concept

Primary data is what *you* (or your team) collected for this question: your survey, your sensor, your lab. You know the bruises. **Secondary** data was collected by someone else for their question: census tables, Kaggle CSVs, a hospital archive. You inherit their bruises without the story. **Tertiary** data is already-summarised material — a review article, a dashboard screenshot, a Wikipedia table of “average rainfall”. You are now two compressions away from reality.

Watch Out

Secondary is not “worse”. Almost all class datasets are secondary. The sin is pretending you collected them. Write down the source, the year, the unit, and who is missing (no phones $\Rightarrow$ no young rural respondents, etc.).

How to write it in Python

```

1 # Always store provenance next to the file, not in your head
2 meta = {
3     "source": "primary",           # or secondary / tertiary
4     "collector": "IT462 lab, 2026",
5     "unit_age": "years",
6     "unit_IT462": "marks / 100",
7 }
8 pd.Series(meta).to_json("data/meta.json", indent=2)

```

1.5 File formats you will actually open**From the class board**

Data $\rightarrow$ file formats: .csv, .xlsx (checked). Branch: .json, .html, webscraping, Python. Separate oval: text.

In Plain Words

A **CSV** is a text file where commas (or tabs) pretend to be columns. An **xlsx** is a zipped XML workbook — Excel’s native form. **JSON** is nested boxes of key-value pairs, the internet’s favourite. **HTML** is a web page; scraping is reading its tables with code instead of with your eyes. **Text** is a novel, a chat log, a statute — unstructured on purpose.

How to write it in Python

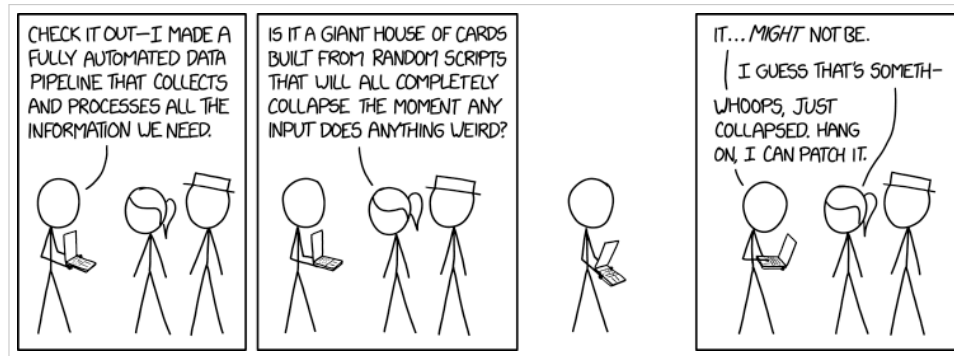
```

1 df = pd.read_csv("marks.csv")           # the default
2 xlsx = pd.read_excel("marks.xlsx")      # needs openpyxl
3 js = pd.read_json("marks.json")
4 html_tables = pd.read_html("https://example.org/results") # list of
   DataFrames
5
6 # 2026: if the CSV is huge, do not start pandas-first
7 # import polars as pl
8 # df = pl.scan_csv("marks.csv").head(5).collect()

```

Watch Out

`pd.read_html` is not “webscraping the interesting bits of the internet”. Many sites forbid it in `robots.txt`, wrap tables in JavaScript, or serve garbage to bots. For this course, scrape only pages you are allowed to scrape. The board circled Python because the *skill* is the parser, not the heist.



xkcd #2054 “Data Pipeline” by Randall Munroe — <https://xkcd.com/2054/> (CC BY-NC 2.5).

Your pipeline is: file → table → IDA → EDA. Everything else is a branch line.

1.6 IDA: making the pile usable

From the class board

Statistics = a tools package aimed at Data. Goal: **usable** — validity, reliability, authenticity, error-free. The name of this step: **IDA**, Initial Data Analysis.

Concept

Four words from the board, in working English:

- **Validity.** Are you measuring the thing you named? A column called **age** that contains 0, 10, 20, 100 might be valid ages, or it might be a missing-code disaster.
- **Reliability.** If you measured again, would you get close to the same number? A bathroom scale that jumps 3 kg every step is valid-ish (it is still “weight”) and unreliable.
- **Authenticity.** Is this the real table, from the real process, not a copy-of-a-copy with a formula paste error?
- **Error-free.** No, it will not be. The phrase means *errors have been hunted*, not *errors do not exist*.

IDA is the hunt. EDA is the looking-for-shape that you are allowed to do *after* the hunt, or interleaved with it — the next section draws the pipeline.

Gujarat-Style Analogy

Before a Navratri garba night you check: is this the actual ticket (authenticity), does the QR still scan (validity), would a second scanner agree (reliability), and did anyone type the date as 31/09 (error). That queue is IDA. EDA is noticing that 80% of the crowd always arrives after 9.

1.7 The dirty table from the board

From the class board

A five-row teaching table — ID, IT462, Age, Gender — deliberately broken.

ID	IT462	Age	Gender
2026.	10	0	M
2026	50	10	F
...	90	20	M
(blank)	−20	100	F
2023	17	(missing)	(missing)

Figure — Reproduce this table in pandas on day one. Every later chapter will abuse it.

Concept

What IDA must scream, row by row:

- 2026. vs 2026: the same ID with a trailing dot. IDs are *nominal labels*, not numbers you average (Chapter 2). Store them as strings.
- Duplicate ID 2026 twice: two people, or one person typed twice?
- Ellipsis and blanks: missingness, not the number zero.
- IT462 = −20: a score below 0 is usually a code for “not attempted” or a sign error. It is not a very humble student.
- Age 0 and 100: 0 can be a newborn (ratio scale, Chapter 2) or a missing code. 100 can be real or a sentinel. You cannot know from the table alone — you need the codebook.
- Gender missing: a categorical hole. Do not fill it with the mode and walk away without a flag column.

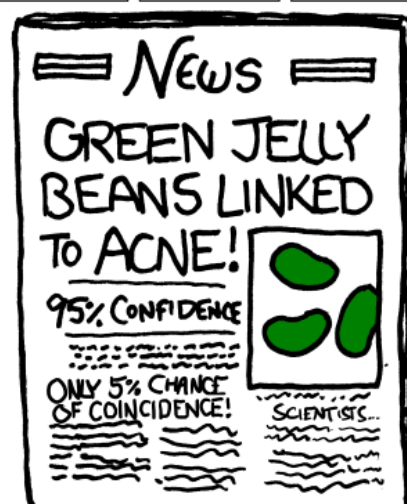
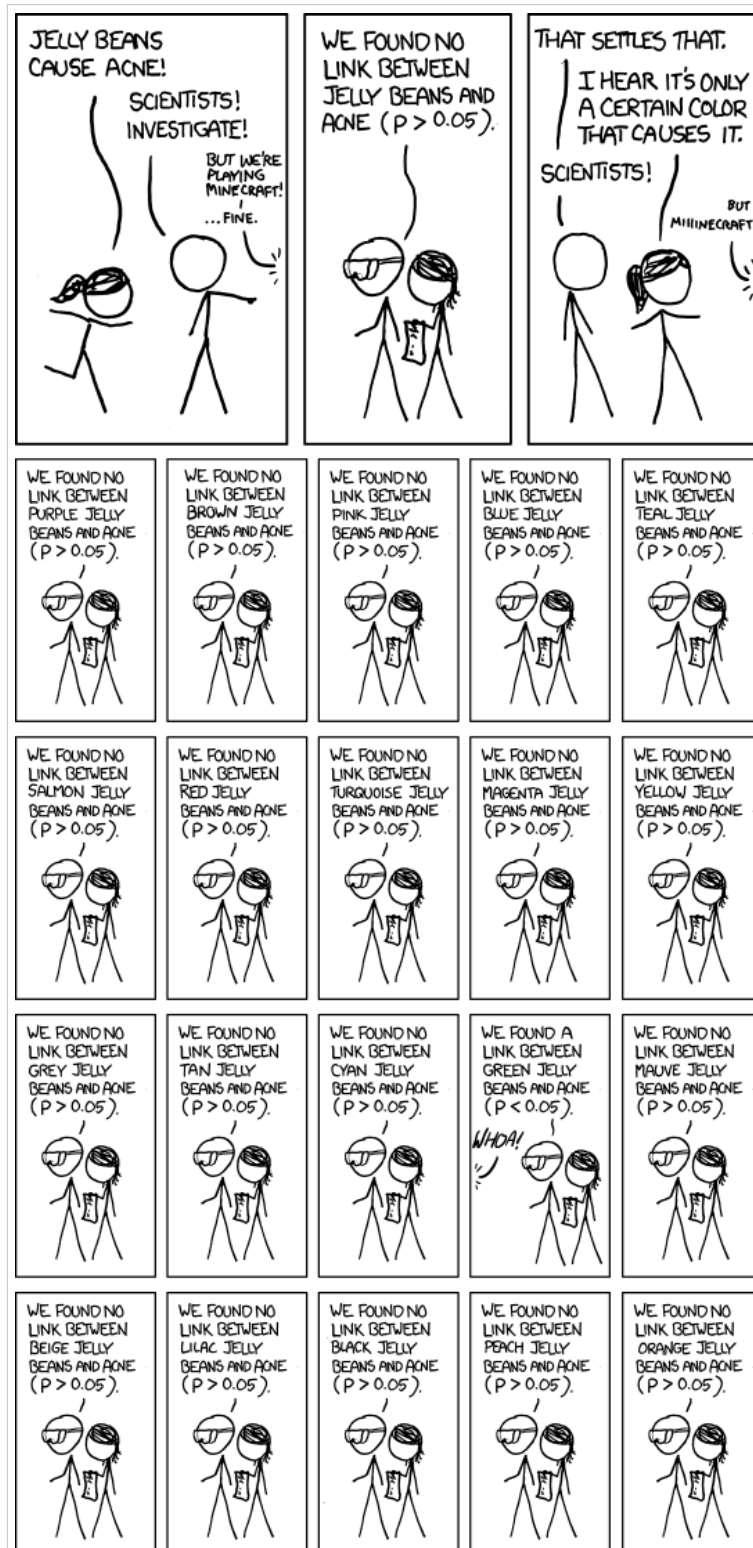
How to write it in Python

```

1 df = pd.DataFrame({
2     "ID":      ["2026.", "2026", None, None, "2023"],
3     "IT462":   [10, 50, 90, -20, 17],
4     "Age":     [0, 10, 20, 100, np.nan],
5     "Gender":  ["M", "F", "M", "F", np.nan],
6 })
7 df["ID"] = df["ID"].astype("string").str.replace(r"\.$", "", regex=
    True)
8 print(df.duplicated(subset=["ID"], keep=False))
9 print((df["IT462"] < 0) | (df["Age"] < 0) | (df["Age"] > 90))
10 print(df.isna().sum())

```

Each line is one of the board’s bruises, turned into a boolean mask. IDA is mostly boolean masks plus a codebook.



xkcd #882 “Significant” by Randall Munroe — <https://xkcd.com/882/> (CC BY-NC 2.5).

Dirty data will happily give you $p < 0.05$. Cleaning is not optional piety; it is how you stop lying.

1.8 IDA, then EDA, then CDA

From the class board

Vertical flow in red: IDA → EDA → CDA. IDA ↔ Problem, Data collection, (Preprocessing) cleaning. EDA ↔ statistical aspects, features of data, variables (with a feedback arrow). CDA ↔ modelling, evaluation.



Figure — The semester in three boxes. Feedback from variables back into IDA is not failure — it is the job.

Concept

IDA (Initial Data Analysis) is “can I trust this rectangle?” Problem definition, collection, cleaning. **EDA** (Exploratory Data Analysis, Tukey) is “what is in here?” — distributions, weird holes, relationships, the features and variables the board circled. **CDA** (Confirmatory Data Analysis) is “does the claim survive a pre-registered test?” — modelling and evaluation. Mixing EDA and CDA on the *same* split is how you overfit a story. Look on train (or on a sandbox). Confirm on a held-out piece you did not stare at.

Watch Out

The feedback arrow on the board is honest: discovering that “Gender” is stored as M/F/male/Female/0/1 sends you back to IDA. That is not going in circles. That is the loop.

1.9 The anatomy of a structured table

From the class board

Structured data: columns = variables or features. Rows = observations or values. Then a fork: categorical (qualitative) vs numerical (quantitative).

In Plain Words

One **row** is one thing in the world: one student, one day, one transaction. One **column** is one question you asked of every thing: age, city, marks. Machine learning people say **feature** for column and **example** for row. Statisticians say **variable** and **observation**. Same rectangle, two dialects. This book uses them interchangeably and will say so when a distinction matters.

columns = features / variables

rows = observations	ID	IT462	Age	Gender
	2026	50	10	F

Figure — Every later formula is a computation down a column, or across two columns.

How to write it in Python

```
1 df.shape           # (n_rows, n_cols) -- lecture 10 will put this
   first
2 df.columns         # the feature names
3 df.index           # the row labels (often 0..n-1, sometimes an ID)
4 df.dtypes          # the computer's guess at measurement scale (it is
   often wrong)
```

`dtypes` saying `int64` does not mean Ratio. Zip codes are integers and nominal. The *scale* is a human decision. That is the entire next chapter.

Questions & Variations

Q. Is a student ID numerical because it is made of digits?

A. No. You cannot add two roll numbers and get a meaningful third student. IDs are nominal labels that happen to be written with digits. Store them as strings.

Q. Where does web-scraped HTML sit in the unstructured / structured split?

A. The page is unstructured (or semi-structured). The table you extract is structured. The act of extraction is IDA, not EDA.

Q. Name Tukey's opposite number — the confirmatory tradition.

A. CDA: hypothesis tests, confidence intervals, pre-specified models. Tukey (1977) argued we had over-invested in CDA and under-invested in looking. 2026's LLM-EDA papers are, ironically, repeating his warning: look, but do not outsource the looking to a black box you cannot audit.

Chapter 2

Measurement Scales: NOIR

This is the spine of the course. If you remember one chapter in a viva, remember this one. Every plot, every mean, every correlation coefficient is legal or illegal *because of the scale of the column*.

2.1 The fork, then the ladder

From the class board

Measurement scales. Categorical $\rightarrow$ Nominal (names, indexes, tags) / Ordinal. Potato: colour = nominal, size = ordinal. Numerical $\rightarrow$ Interval / Ratio. Side mnemonic: N–O–I–R.

In Plain Words

First, a binary question: is this column a **label** or a **quantity**? Labels are **categorical** / **qualitative**. Quantities are **numerical** / **quantitative**. Then each side splits in two, and the four children have a famous acronym: **NOIR** — Nominal, Ordinal, Interval, Ratio — in increasing order of mathematical power.

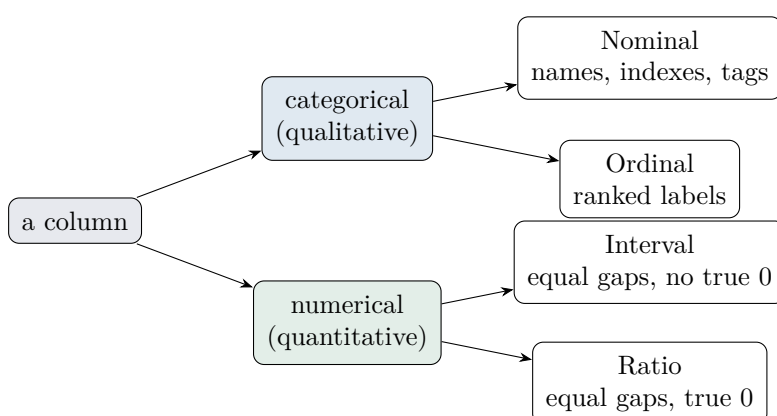


Figure — The board's tree, redrawn. You walk top-down with the decision questions of Lecture 6.

Gujarat-Style Analogy

A potato is a teaching device, not a grocery list. **Colour** (red / yellow / white) is a name — nominal. **Size** (S / M / L / XL) is a rank — ordinal. **Weight in grams** is a ratio: 0 g means no potato, and 400 g is twice 200 g. **Storage temperature in °C** is interval:

20°C is not “twice as hot” as 10°C, because 0°C is not the absence of temperature.

2.2 The property table (memorise this)

From the class board

Four rows of properties, four columns N O I R, ticks accumulating as you walk right: labelled; inherent order; measurable difference; absolute (true) zero.

property	N	O	I	R
labelled / category exists	✓	✓	✓	✓
inherent order	×	✓	✓	✓
measurable equal difference	×	×	✓	✓
absolute (true) zero	×	×	×	✓

Figure — Each level keeps everything to its left and gains one new power. Ratio is Nominal+order+equal gaps+true zero.

Concept

Nominal. Name only. Blood group, city, gender, roll number, colour of potato. You may count, tabulate, compute mode and proportions. You may not rank, subtract, or average.

Ordinal. Name plus order. Likert items, education level, spicy-level of the thaali (mild < medium < extra), T-shirt size. You may rank. You may not honestly claim that the gap mild→medium equals medium→extra.

Interval. Order plus equal gaps, but 0 is a convention, not “nothing”. Calendar year, Celsius, Fahrenheit, some psychometric scores after a strong argument. $20^{\circ}\text{C} - 10^{\circ}\text{C} = 10^{\circ}\text{C} - 0^{\circ}\text{C}$, but 20°C is not twice 10°C .

Ratio. Interval plus a true zero: zero means none of the thing. Age, length, mass, Kelvin, income, counts, time duration, marks-out-of-100 if 0 means “no marks”. Twice is allowed: 10 years is twice 5 years.

2.3 Nominal, slowly

In Plain Words

A nominal variable partitions the world into buckets that refuse to line up. “Ahmedabad” is not less than “Surat”. Coding them as 1 and 2 is a storage trick, not maths.

How to write it in Python

```
1 s = pd.Series(["M", "F", "M", "F", None], dtype="string")
2 print(s.value_counts(dropna=False))           # frequency --- LEGAL
3 print(s.value_counts(normalize=True))         # proportion / %
4 print(s.mode())                               # the only "average"
   you may quote
5 # s.mean() --> TypeError or nonsense if you had coded M=1, F=0 and
```

```

        forgotten
6  s_cat = s.astype("category")           # tells pandas: this is
        nominal
7  print(pd.get_dummies(s_cat, dummy_na=True)) # one-hot: the ML
        encoding

```

One-hot encoding does not smuggle order in. That is why it is the default for nominal features going into a model. Ordinal encoding (S=1,M=2,L=3) *does* smuggle order — legal for ordinal, a lie for city names.

Watch Out

pandas will cheerfully compute `df['zipcode'].mean()`. The computer does not know NOIR. You do.

2.4 Ordinal, and the Likert scale

From the class board

Likert scale. Five ticks: strongly disagree, disagree, neutral, agree, strongly agree. Side note: 3-point, 5-point, 7-point variants.

In Plain Words

A **Likert item** is a sentence (“the canteen sambhar is reliable”) plus an ordered row of agreement. 3-point is coarse (disagree / neutral / agree). 5-point is the board’s drawing. 7-point adds “slightly” on each side and is loved by survey people who want more resolution and feared by students who then treat 1..7 as centimetres.

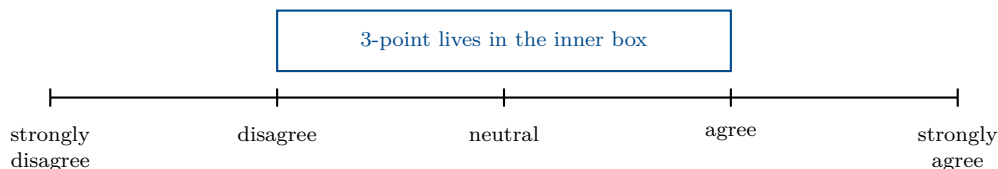


Figure — The board’s 5-point axis. A 3-point scale is the inner three labels; a 7-point scale inserts two more ticks.

How to write it in Python

```

1  likert = pd.Categorical(
2      ["disagree", "agree", "neutral", "strongly agree", "disagree"],
3      categories=["strongly disagree", "disagree", "neutral",
4                  "agree", "strongly agree"],
5      ordered=True,           # THIS is what makes it ordinal
6  )
7  print(likert.sort_values())
8  print(likert.min(), likert.max())           # legal: order exists
9  # print(likert.mean())                     # illegal as
        categorical
10 codes = likert.codes                     # 0..4 integers,
        tempting

```

`ordered=True` is the entire ordinal idea in software. Without it, pandas thinks “agree” and “disagree” are just two city names.

2.5 The board’s transform: ordinal $\rightarrow$ interval via Likert

From the class board

Arrow: Ordinal $\xrightarrow{\text{transform, Likert scale}}$ Interval. Goals: (1) make quantifiable equivalence with numbers; (2) make the difference equal in length.

Concept

This is the most philosophically loaded arrow on the whole semester’s boards, so we will not rush it.

What the class is doing. You print 1, 2, 3, 4, 5 under the five Likert labels and then treat those integers as if the mental gap “disagree $\rightarrow$ neutral” equalled “neutral $\rightarrow$ agree”. Once you accept equal length, addition and subtraction become legal, and you may compute a mean of 3.4 “agreement points”.

What a careful statistician adds. Stevens (1946), who named NOIR, would call this a *convention*, not a discovery. Many working scientists do it anyway, especially when they average *several* Likert items into a scale (Likert *scale* originally meant the sum of items, not one item). The honest move in 2026:

- For **one** item: prefer median, mode, bar chart of counts. Spearman, not Pearson (Chapter 4).
- For a **sum of many** well-designed items: treating the sum as interval is a standard, argued, and imperfect practice. Report both the median and the mean. Look at the histogram. Do not write “people are $1.37\times$ more agreed”.

Derivation — step by step

Suppose we map

$$\text{SD} \mapsto 1, \text{D} \mapsto 2, \text{N} \mapsto 3, \text{A} \mapsto 4, \text{SA} \mapsto 5.$$

The board’s “equal in length” claim is

$$(2 - 1) = (3 - 2) = (4 - 3) = (5 - 4) = 1.$$

That equation is true of the *numbers we wrote*. It is an assumption about the *minds we surveyed*. If “neutral $\rightarrow$ agree” is a bigger psychological jump than “agree $\rightarrow$ strongly agree” (it often is — people are shy of extremes), the equation is false in the world and true on the page. EDA’s job is to look at the histogram of the mapped numbers: a pile-up on 3 and 4 with almost no 1s is a hint that the equal-gap story is strained.

Watch Out

Never map ordinal to interval and then compute ratios. “Agree (4) is twice disagree (2)” is a crime even after the board’s transform. The transform, at best, licenses $+$ and $-$. $\times$ and $\div$ still want a true zero, which “strongly disagree” is not.

2.6 Interval vs ratio, and why age is ratio

From the class board

Age — Ratio (because of Absolute Zero). Parenthetical joke/note: “a person with age 0 is impossible”. Variables / features hold values.

In Plain Words

Absolute zero means the number 0 on this scale is the absence of the quantity. Age from birth: 0 years is “just born”, not “we have no idea”. You cannot be -3 years old. A 20-year-old has lived twice a 10-year-old’s life. That is the ratio license.

Concept

The board’s quip “a person with age 0 is impossible” is a biological aside (you meet students, not newborns) and must not be confused with the scale. The *scale* has a true zero even if your *sample* does not contain it. Income of 0 is possible; age of 0 is possible; temperature of 0°C is *not* “no temperature”, which is why Celsius is interval and Kelvin is ratio.

Derivation — step by step

Let a and b be ages in years. Ratio statements are the numbers a/b .

$$\frac{20}{10} = 2 \quad \text{means “twice as old”}.$$

For Celsius,

$$\frac{20^{\circ}\text{C}}{10^{\circ}\text{C}} = 2$$

does *not* mean twice as hot. Convert to Kelvin ($T_K = T_C + 273.15$):

$$\frac{293.15}{283.15} \approx 1.035,$$

about 3.5% hotter on an absolute scale. Same arithmetic, different license. That is NOIR in one calculation.

How to write it in Python

```
1 age = pd.Series([0, 10, 20, 100, np.nan])
2 print(age.min(), age.max())           # feasible range --- lecture
   10
3 print((age / age.median()).describe()) # ratio statements: legal
   here
4 celsius = pd.Series([0, 10, 20, 100])
5 # (celsius / celsius.median()) is legal as arithmetic and illegal
   as physics
```

2.7 Which operations are legal?

From the class board

Operations table: N $\times$, O $\times$, I + and $-$, R $\times$ and $\div$.

	Nominal	Ordinal	Interval	Ratio
$= \neq$ (same / not same)	yes	yes	yes	yes
$< >$ (rank)	no	yes	yes	yes
$+ -$ (gap)	no	no*	yes	yes
$\times \div$ (ratio)	no	no	no	yes
a typical centre	mode	median	mean [†]	mean
a typical spread	counts	IQR / range of ranks	SD [†]	SD, CV

*some people subtract Likert codes anyway — see previous section. [†]after the Likert-as-interval convention, or for genuine interval data.

In Plain Words

The board drew a big X under N and O because you do not *add genders* or *multiply spicy-levels*. Under I it wrote + and $-$. Under R it wrote $\times$ and $\div$ — not because you *cannot* add masses, but because the new power that appears at R is the ratio.

2.8 The decision tree from Lecture 6

From the class board

Dataset variables, measurement scales: (1) Can I count / identify group? $\rightarrow$ Nominal. Ops: %, frequency, counts, proportions. (2) Can I rank? $\rightarrow$ Ordinal. (3) Can I measure the difference? $\rightarrow$ Interval. Ops: +, $-$. (4) Can I find the ratio? $\rightarrow$ Ratio. Ops: $\times$, $\div$.

Derivation — step by step

Walk a mystery column X from the top. Stop at the first “no”.

1. If the only honest verb is “belongs to group ...” $\rightarrow$ **Nominal**. Compute n_g/n for each group g .
2. Else, if you can sort every pair but not measure a gap $\rightarrow$ **Ordinal**.
3. Else, if gaps are equal but $X = 0$ does not mean “none” $\rightarrow$ **Interval**. Legal: $\bar{x}$, $x_i - x_j$, SD.
4. Else, 0 is nothing and $2x$ is twice $\rightarrow$ **Ratio**. Legal: also x_i/x_j and the coefficient of variation $s/\bar{x}$.

Worked identifications from this course:

- **Gender** $\rightarrow$ stop at (1) $\rightarrow$ Nominal.
- Likert item $\rightarrow$ pass (1), stop at (2) $\rightarrow$ Ordinal (then optionally *treat* as interval).
- IT462 marks out of 100 $\rightarrow$ pass (1)–(3), 0 marks is none $\rightarrow$ Ratio.

- Age → Ratio (board).
- ID → Nominal, even if it looks like 202611032.
- Year of birth → Interval (the year 0 is a calendar convention). Age derived from it → Ratio.

How to write it in Python

```

1 def guess_scale(s: pd.Series) -> str:
2     """A helper, not an oracle. Humans still own NOIR."""
3     if s.dtype == "boolean" or set(s.dropna().unique()) <= {0, 1, "0", "1", "M", "F"}:
4         return "nominal-or-binary (check codebook)"
5     if isinstance(s.dtype, pd.CategoricalDtype):
6         return "ordinal" if s.dtype.ordered else "nominal"
7     if pd.api.types.is_numeric_dtype(s):
8         if (s.dropna() >= 0).all() and s.min() == 0 or "age" in str(s.name).lower():
9             return "ratio-candidate"
10        return "interval-or-ratio (ask: is 0 nothing?)"
11    return "nominal (text)"
12
13 for col in df.columns:
14     print(col, guess_scale(df[col]))

```

Notice the function is full of hedges. That is correct behaviour. Lecture 10 will tell you to check **type (measurement scales)** as a human, right after `df.shape`.

Questions & Variations

Q. Why is a count of students in a class ratio, not just interval?

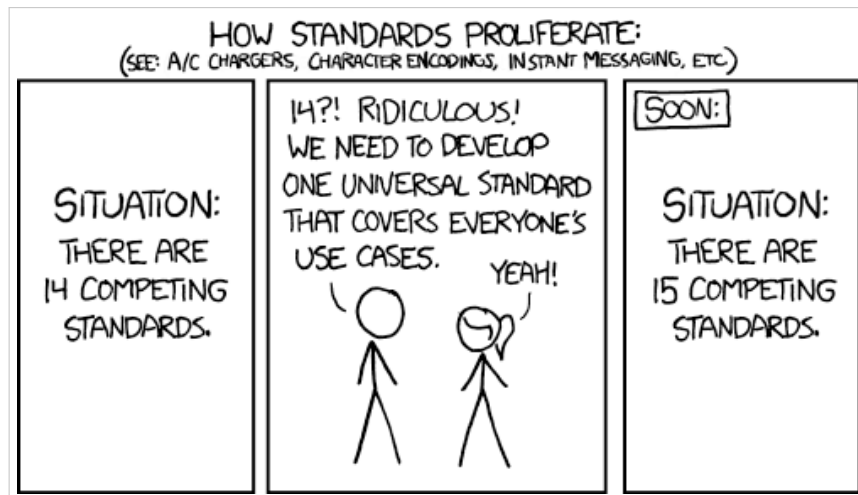
A. Zero students is none. Forty is twice twenty. Counts are the poster-child of ratio.

Q. Is GPS latitude ratio?

A. Interval-ish: 0° is a convention (the equator), not “no latitude”. Differences are meaningful (kilometres north). Ratios are not.

Q. Can I one-hot encode ordinal?

A. You can, and you throw the order in the bin. Sometimes that is safer than pretending gaps are equal. Sometimes (trees, linear models with a monotone effect) the ordered codes are better. EDA: plot both.



xkcd #927 "Standards" by Randall Munroe — <https://xkcd.com/927/> (CC BY-NC 2.5).

NOIR is the one standard that actually prevents you from inventing a fourteenth way to average city names.

Chapter 3

Central Tendency and Dispersion

From the class board

Statistical tools split into **Descriptive** and **Inferential**. Descriptive = summary / description of data. “5 different measures”. Central tendency: mean, median, mode.

In Plain Words

Descriptive statistics compress *this* table. **Inferential** statistics use this table to say something about a larger world (Chapter 5). EDA is mostly descriptive, with the humility to remember the world is bigger. The board’s “five measures” are the families we will now fill: central tendency, dispersion, relative standing (percentiles), shape (skewness / kurtosis), and association. This chapter does the first two.

3.1 The centre: three pins, not one

From the class board

Measures of central tendency → Mean (Arithmetic, Geometric, Harmonic), Median, Mode. Table: mean does not need a sort, $\bar{x} = \sum x_i / n$, plus trimmed and weighted variants. Median needs a sort; odd / even formulae. Mode needs a sort (practically). Tiny example: 1, 2, 4, 6.

3.1.1 Arithmetic mean

Derivation — step by step

For a sample $x_1, \dots, x_n$,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

You do *not* sort. Every value, however wild, pulls the pin. That is the mean’s superpower and its curse.

Why divide by n ? $\sum x_i$ grows with the pile size. Dividing by n makes “average marks” comparable between a class of 4 and a class of 400.

The balancing-property. The signed deviations sum to zero:

$$\sum_{i=1}^n (x_i - \bar{x}) = \sum x_i - n\bar{x} = \sum x_i - n \cdot \frac{\sum x_i}{n} = 0.$$

The mean is the unique number about which the table is in torque-balance. On a number line, it is the centre of mass if each point has mass 1.

Worked board example. $x = (1, 2, 4, 6)$, $n = 4$:

$$\bar{x} = \frac{1 + 2 + 4 + 6}{4} = \frac{13}{4} = 3.25.$$

Concept

Trimmed mean. Sort, throw away the lowest $p\%$ and the highest $p\%$, then take the arithmetic mean of what remains. The board: “mean after removing the extremes x_i ”. A 10% trimmed mean on 100 numbers drops the 10 smallest and 10 largest. It is the adult who has met outliers and would still like a mean.

Weighted mean. If value x_i has weight $w_i > 0$ (a frequency, a reliability, a mass),

$$\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}.$$

The ordinary mean is the special case $w_i = 1$. Credit-GPA is a weighted mean of grade-points by credits.

How to write it in Python

```
1 x = np.array([1., 2, 4, 6])
2 print(x.mean()) # 3.25
3 print(np.average(x, weights=[1, 1, 1, 3])) # 4.0 -- 6 counts
   three times
4 from scipy.stats import trim_mean
5 print(trim_mean(x, 0.25)) # drop 25% each side ->
   mean(2,4)=3
```

3.1.2 Geometric and harmonic means

Derivation — step by step

For $x_i > 0$,

$$\text{GM} = \left(\prod_{i=1}^n x_i \right)^{1/n} = \exp\left(\frac{1}{n} \sum_{i=1}^n \log x_i\right), \quad \text{HM} = \frac{n}{\sum_{i=1}^n 1/x_i}.$$

The second form of GM is how computers avoid overflow (multiply 200 growth-rates directly and you explode; average their logs and **exp** back).

AM–GM–HM inequality. For positive numbers,

$$\text{AM} \geq \text{GM} \geq \text{HM},$$

with equality if and only if every x_i is equal. On the board's $\{1, 2, 4, 6\}$:

$$\text{AM} = 3.25, \quad \text{GM} = (48)^{1/4} \approx 2.632, \quad \text{HM} = \frac{4}{1 + 1/2 + 1/4 + 1/6} = \frac{4}{1.9167} \approx 2.087.$$

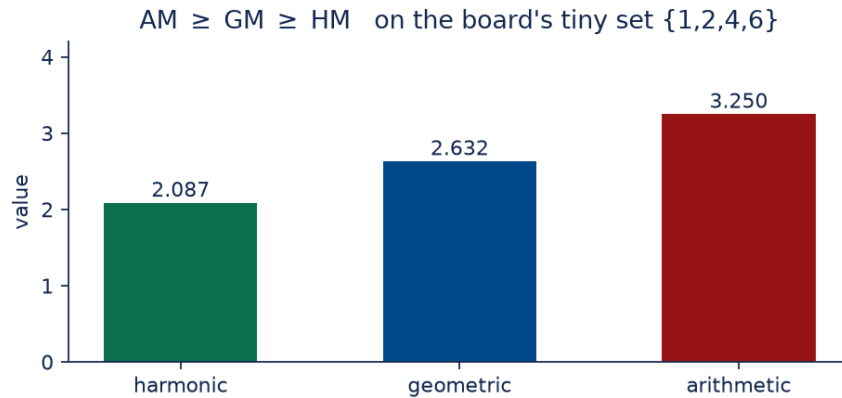


Figure — The inequality is not a vibe. It is a theorem, and the board's four numbers already show it.

Concept

When to use which. AM — additive stuff (marks, heights, kilograms). GM — multiplicative stuff (growth rates, returns, fold-changes). Average a 10% up year and a 10% down year with AM and you will think you broke even; GM tells the truth (0.99). HM — rates with a fixed numerator (average speed over equal *distances*, parallel resistances). If you drive 60 km at 30 km/h and 60 km at 90 km/h, your average speed is the harmonic mean 45, not 60.

How to write it in Python

```
1 from scipy.stats import gmean, hmean
2 print(gmean(x), hmean(x))
```

3.1.3 Median

Derivation — step by step

Sort the sample into $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$. The parentheses mean **order statistics**: $x_{(1)}$ is the smallest, $x_{(n)}$ the largest.

If n is odd, the median is the middle order statistic

$$\tilde{x} = x_{\left(\frac{n+1}{2}\right)}.$$

If n is even there is no single middle index, so the board averages the two central ones:

$$\tilde{x} = \frac{1}{2} \left(x_{(n/2)} + x_{(n/2+1)} \right).$$

Worked. $n = 4$ even, sorted already $(1, 2, 4, 6)$:

$$\tilde{x} = \frac{1}{2} (x_{(2)} + x_{(3)}) = \frac{1}{2} (2 + 4) = 3.$$

For $n = 5$, say $(1, 2, 4, 6, 8)$: $\frac{5+1}{2} = 3$, median = $x_{(3)} = 4$.

In Plain Words

The median is the 50th percentile (next chapter). Half the ordered table is below it, half above. Changing the largest value to a billion does not move it — unlike the mean. That is why we quote medians for income, house prices, and anything a billionaire can sit in.

How to write it in Python

```
1 print(np.median(x))           # 3.0 (sorts for you)
2 print(np.median([1, 2, 4, 6, 8])) # 4.0
```

The board’s “sort required: yes” is exactly this: there is no median formula that avoids order. `numpy` just hides the sort.

3.1.4 Mode

In Plain Words

The mode is the value that appears most often. A table can have one mode (unimodal), two (bimodal), many, or — if every value is unique — none that matters. It is the only centre that is legal for **nominal** data: the most common blood group, the most common city.

How to write it in Python

```
1 print(pd.Series([1, 2, 2, 4, 6]).mode().tolist()) # [2]
2 print(pd.Series(["M", "F", "M", "F", "M"]).mode()) # M
```

Gujarat-Style Analogy

At a farsan counter the **mode** is whatever sold most (gathiya, obviously). The **median** bill is the middle customer. The **mean** bill is pulled up by the one uncle who bought for a wedding. Quote all three and you understand the evening. Quote only the mean and you will overstock the wedding tray.

Watch Out

Scale $\leftrightarrow$ centre, as the board promised (“statistical measures for measurement scales — completed”): nominal $\rightarrow$ mode; ordinal $\rightarrow$ median (mode ok); interval / ratio $\rightarrow$ mean if the shape is not feral, median if it is.

3.2 Dispersion: how wide is the pile?

From the class board

Measures of dispersion / variation. $x_1, \dots, x_n$. Range = $x_H - x_L$ (max – min). Population variance $\sum(x_i - \bar{x})^2/N$. Sample variance $\sum(x_i - \bar{x})^2/(n - 1)$ (ticked). Standard deviation = $\sqrt{\text{variance}}$.

3.2.1 Range

Derivation — step by step

$$\text{Range} = x_{(n)} - x_{(1)} = x_H - x_L.$$

On (1, 2, 4, 6): $6 - 1 = 5$. On the percentile data of the next chapter: $20 - 2 = 18$. One outlier sets both ends on fire. Range is a first glance, never a last.

3.2.2 Variance: the average squared red length

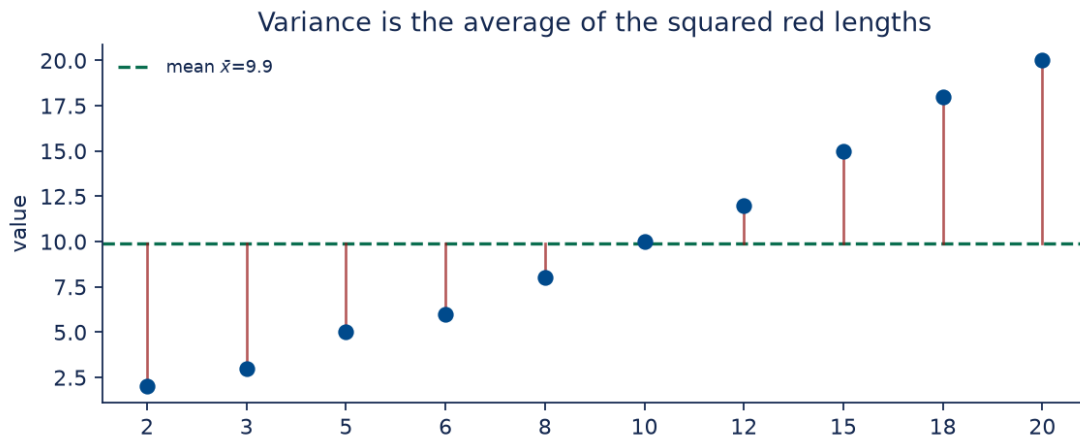


Figure — Each red stick is $x_i - \bar{x}$. Variance squares those sticks (so that down-sticks do not cancel up-sticks) and averages.

Derivation — step by step

Start from “how far is each point from the mean?” The raw deviations $\sum(x_i - \bar{x})$ are *always* 0 — we proved it. Useless as a spread.

Step 1 — kill the sign by squaring:

$$d_i^2 = (x_i - \bar{x})^2 \geq 0.$$

Step 2 — average the squares. If you have the *whole* population of size N ,

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2,$$

where μ is the population mean (a parameter, Chapter 5). This is the board’s **Popⁿ** formula.

If you have a *sample* of size n and you had to *estimate* μ by $\bar{x}$,

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

This is the board’s ticked **Sample** formula.

Why $n - 1$? Bessel’s correction, slowly. $\bar{x}$ is computed from the same n numbers. That uses up one piece of freedom: once you know $\bar{x}$ and $n - 1$ of the values, the last value is forced (because deviations sum to 0). Equivalently: $\bar{x}$ sits *inside* the sample, so

the deviations from $\bar{x}$ are a bit too small compared with deviations from the unknown μ . Dividing by $n - 1$ instead of n inflates s^2 just enough to make

$$\mathbb{E}[s^2] = \sigma^2$$

for independent draws from a population with finite variance. Dividing by n would give a pessimistic, downward-biased estimate of the population spread.

Worked. $x = (1, 2, 4, 6)$, $\bar{x} = 3.25$:

$$\begin{aligned}\sum (x_i - \bar{x})^2 &= (1 - 3.25)^2 + (2 - 3.25)^2 + (4 - 3.25)^2 + (6 - 3.25)^2 \\ &= 5.0625 + 1.5625 + 0.5625 + 7.5625 = 14.75.\end{aligned}$$

Population style (if these four numbers *are* the world): $14.75/4 = 3.6875$. Sample style (if they are a sample): $14.75/3 \approx 4.917$.

How to write it in Python

```
1 print(np.var(x, ddof=0))    # /n      population --- numpy default
2 print(np.var(x, ddof=1))    # /(n-1) sample --- pandas default
3 print(pd.Series(x).var())   # 4.9166...
```

This is a famous trap. `numpy.var` divides by n . `pandas.Series.var` divides by $n - 1$. The board ticked the sample formula, which is the pandas one. Always pass `ddof=` explicitly in mixed code.

3.2.3 Standard deviation

Derivation — step by step

Squares have square units: marks², years². Nobody thinks in those. Take the square root, as the board drew:

$$s = \sqrt{s^2}, \quad \sigma = \sqrt{\sigma^2}.$$

Now the spread is back in the original unit. For the worked sample, $s \approx 2.217$ marks (or whatever x was).

Concept

Two cousins you should know even though they were not named on the board:

$$\text{IQR} = Q_3 - Q_1 \quad (\text{next chapter}), \quad \text{MAD} = \text{median}_i |x_i - \tilde{x}|,$$

and the **coefficient of variation** (ratio-scale only)

$$\text{CV} = \frac{s}{\bar{x}}.$$

CV lets you compare the spread of heights to the spread of incomes, because it is unitless. It is illegal for Celsius (interval) for the same reason ratios are.

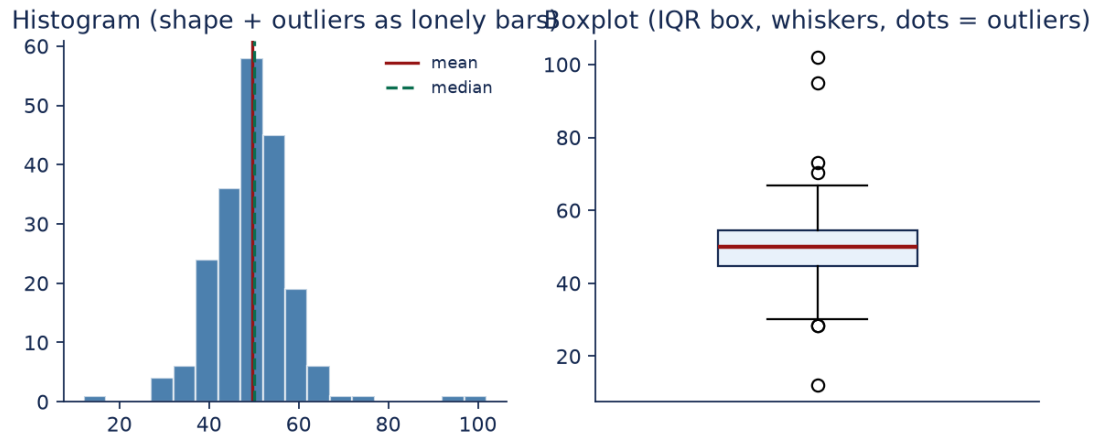


Figure — A histogram shows the shape the variance cannot. A boxplot shows the IQR the range cannot. Use both.

Chai-Break

Variance is the friend who reports every complaint *squared* so that the one person who shouted cannot be cancelled by the one who whispered. SD is that friend after a cup of chai, talking in normal units again.

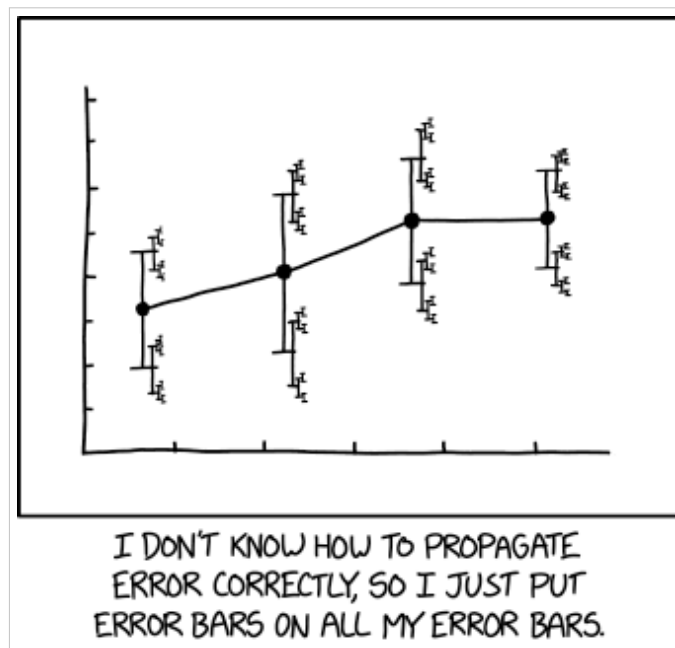
Questions & Variations

Q. Why not average the absolute deviations $\frac{1}{n} \sum |x_i - \bar{x}|$?

A. You can — it is the mean absolute deviation. It is more robust and less algebraically convenient. Squares lead to Pythagoras, Gaussians, and least squares (the FOML book). That is why we live with s^2 .

Q. Does the geometric mean have a “variance”?

A. People use the variance of $\log x_i$. Multiplicative data, logarithmic summaries.



xkcd #2110 “Error Bars” by Randall Munroe — <https://xkcd.com/2110/> (CC BY-NC 2.5).

A mean without a spread is a dare. SD, IQR, or at least a range — pick one and put it next to the pin.

Chapter 4

Percentiles, Shape, and Association

4.1 Relative standing

From the class board

Quartile deviation; percentiles (heavily circled); quartile; quantile; decile. Number line of length n . Percentile position of x_i :

$$P(x_i) = \frac{i-1}{n-1} \times 100 \quad \text{and the alternative} \quad \frac{i}{n} \times 100.$$

In Plain Words

A **percentile** answers “what fraction of the ordered pile is below this?” A **quantile** is the same idea stated as a proportion in $[0, 1]$ instead of a percent in $[0, 100]$. **Quartiles** are the 25th, 50th, 75th percentiles ($Q_1, Q_2 = \text{median}, Q_3$). **Deciles** cut the pile into tenths (D_1 is the 10th percentile). **Quartile deviation** (semi-interquartile range) is

$$\text{QD} = \frac{Q_3 - Q_1}{2}.$$

Derivation — step by step

Sort so that index $i = 1$ is the smallest and $i = n$ the largest. The board’s **main** formula,

$$P(x_{(i)}) = \frac{i-1}{n-1} \times 100,$$

is the “fraction of the way along the index range”. It sends the smallest value to 0% and the largest to 100%. The alternative $i/n \times 100$ sends the largest to $100 \cdot n/n = 100\%$ but the smallest to $100/n$, never zero.

Inverting the main formula to find the index of a desired percentile P :

$$i-1 = \frac{P}{100} (n-1) \quad \Rightarrow \quad i = 1 + \frac{P}{100} (n-1).$$

If i is not an integer, interpolate between the surrounding order statistics.

2026 cutting edge

Hyndman & Fan (1996) catalogued *nine* sample-quantile definitions. NumPy and pandas default to type 7 (linear interpolation of $i = 1 + p(n-1)$ — which *is* the board's inversion). Type 8 is preferred by Hyndman; R's `quantile()` historically defaulted to type 7 too. In a viva: “there is no one percentile formula; we use the class formula $(i-1)/(n-1)$, which matches `numpy` default interpolation.”

4.1.1 The ten numbers from the board**From the class board**

Data 2, 3, 5, 6, 8, 10, 12, 15, 18, 20, $n = 10$. Q1: $P(12)$ with $i = 7 \rightarrow 67\%$. Q2: find the 25th percentile $\rightarrow i = 3.25$.

Derivation — step by step

Sorted, index i under the value:

i	1	2	3	4	5	6	7	8	9	10
$x_{(i)}$	2	3	5	6	8	10	12	15	18	20

Q1 on the board. $x = 12$ sits at $i = 7$:

$$P(12) = \frac{7-1}{10-1} \times 100 = \frac{6}{9} \times 100 = 66.\bar{6}\% \approx 67\%.$$

Q2 on the board. Want $P = 25$:

$$i-1 = \frac{25 \times 9}{100} = 2.25 \quad \Rightarrow \quad i = 3.25.$$

So the 25th percentile sits a quarter of the way from $x_{(3)} = 5$ to $x_{(4)} = 6$:

$$P_{25} = 5 + 0.25 \cdot (6 - 5) = 5.25.$$

(The board stopped at $i = 3.25$; the interpolation is the surrounding step every computer then takes.)

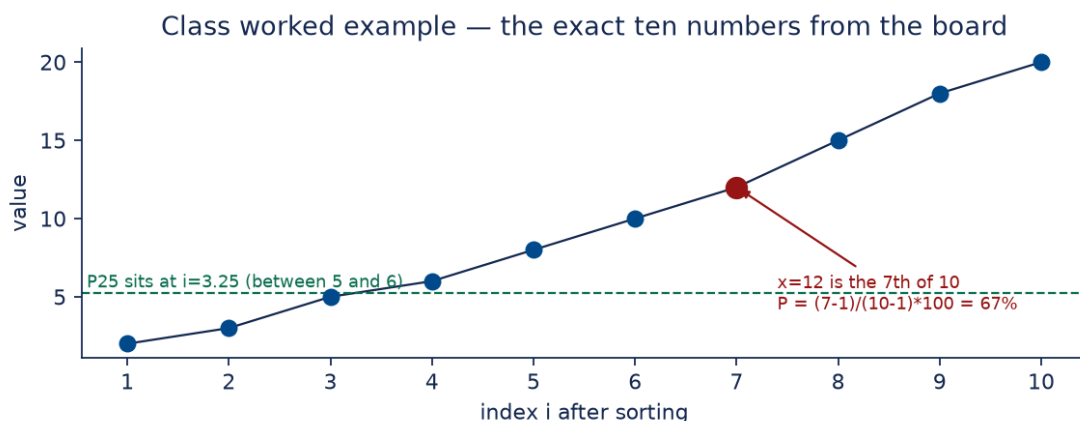


Figure — The exact class numbers. Red: 12 is the 67th percentile. Teal: P_{25} lives between 5 and 6.

How to write it in Python

```

1 x = np.array([2,3,5,6,8,10,12,15,18,20])
2 n, i = len(x), 7
3 print((i-1)/(n-1)*100)                # 66.66...   board: 67%
4 print(np.quantile(x, 0.25))           # 5.25      matches i=3.25
   interpolation
5 print(np.percentile(x, 25))           # same thing, 0-100 scale
6 q1, q3 = np.quantile(x, [0.25, 0.75])
7 print((q3-q1)/2)                     # quartile deviation

```

4.2 Skewness: the pile's lean

From the class board

Statistical measures: CT, dispersion, **Skewness (symmetry)**. Sketches: a right-tailed curve; a bell with Mean, Median, Mode stacked at the centre. `df[''].skew()` in pandas. Extra tents: triangle (symmetric) and a rectangular pulse (uniform).

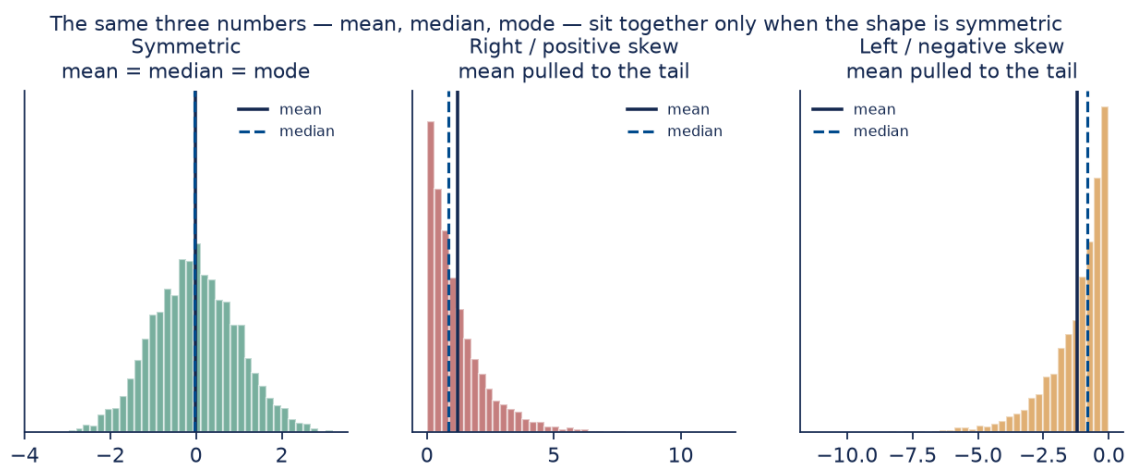


Figure — Symmetric: the three centres coincide. Right skew: the mean is dragged into the long tail. Left skew: the mirror image.

In Plain Words

A pile is **symmetric** if the left of the centre is a mirror of the right. Then mean = median = mode (the board's stacked labels). **Right / positive skew** has a long tail toward large values (income, city house-prices, the 99 in Lecture 1). **Left / negative skew** has a long tail toward small values (easy-exam marks piled near 90, with a few catastrophes).

Derivation — step by step

Two classical formulae; know both, because pandas will pick one for you.

Pearson's moment coefficient of skewness

$$g_1 = \frac{m_3}{s^3}, \quad m_3 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3.$$

Cubes keep a sign: a long right tail makes $m_3 > 0$. Dividing by s^3 makes g_1 unitless, so you can compare skew of kilograms to skew of rupees.

Pearson's median skewness (a robust cousin)

$$\frac{3(\bar{x} - \tilde{x})}{s}.$$

If the mean sits to the right of the median, this is positive — right skew. The board's cartoon is exactly this sentence.

Fisher–Pearson (what `Series.skew()` uses by default) adjusts g_1 for sample size with a factor $n/((n-1)(n-2))$. For $n = 10$ it matters; for $n = 10^5$ it does not.

How to write it in Python

```
1 s = pd.Series([17, 41, 30, 99])           # lecture 1, right-skewed by
    the 99
2 print(s.skew())                          # Fisher-Pearson
3 print(3*(s.mean()-s.median())/s.std())
```

Concept

The board's extra sketches: a **triangular** / **tent** density that is still symmetric (mean = median = mode, but not Gaussian). A **uniform** / **rectangular** density: every value in an interval is equally likely, platykurtic (next section), skewness 0. Symmetry is not “bell-shaped”. Bell-shaped is one symmetric fashion.

4.3 Kurtosis: tailedness / peakedness**From the class board**

(4) Kurtosis (Tailedness / Peakedness). Three bells: leptokurtic (tall), meso, platy (flat).

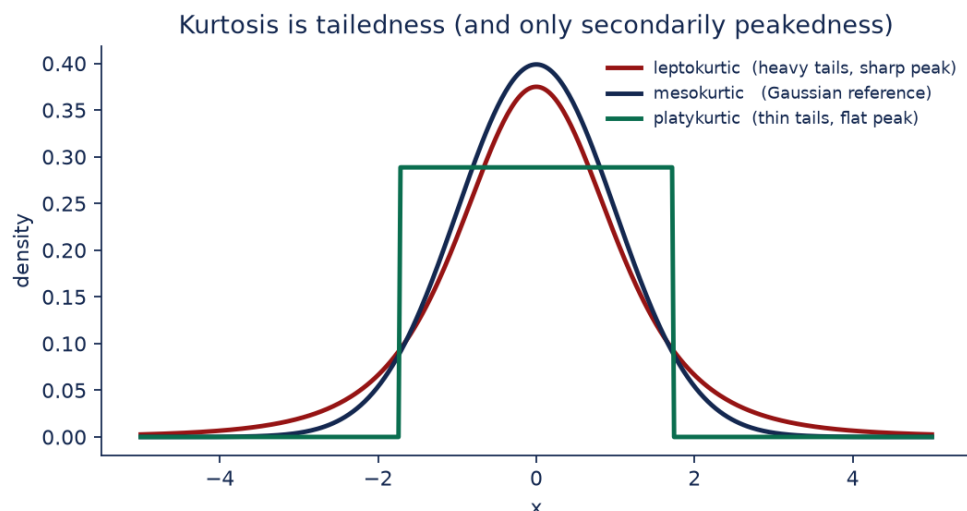


Figure — *Lepto* = heavy tails (and usually a sharp peak). *Meso* = Gaussian reference. *Platy* = thin tails, shoulder-y, like a uniform.

Derivation — step by step

The fourth standardised moment is

$$\text{Kurtosis} = \frac{m_4}{s^4}, \quad m_4 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4.$$

A Gaussian has kurtosis 3. People therefore quote **excess kurtosis**

$$\kappa = \frac{m_4}{s^4} - 3,$$

which is 0 for a perfect bell. Pandas `Series.kurt()` returns excess kurtosis (Fisher). Signs:

- $\kappa > 0$ **leptokurtic** — outliers are more common than a bell would allow (asset returns, the word “lepto” = skinny peak).
- $\kappa = 0$ **mesokurtic** — Gaussian-like tails.
- $\kappa < 0$ **platykurtic** — tails thinner than a bell; the uniform on an interval is the cartoon.

The modern consensus (Westfall 2014, and every 2026 stats blog that is not lying): kurtosis is *primarily about tails*, and peakedness is a side-effect. The board wrote both words; prefer tails when you speak.

How to write it in Python

```
1 print(pd.Series(x).kurt())      # excess kurtosis
2 print(pd.Series(x).kurtosis()) # alias
```

4.4 Association: how X and Y move

From the class board

(5) measures of relationship / association. Variables X, Y . Linear vs non-linear. Scatter plot with a positive ellipse and a major-axis arrow. Negative ellipse plus outliers. Pearson drawn on a boomerang cloud, with `df.corr()`. Method map: interval/ratio $\rightarrow$ Pearson r ; ordinal $\rightarrow$ Spearman r_s ; Y continuous, X binomial $\rightarrow$ point-biserial; binomial $\times$ binomial $\rightarrow$ Cramér ϕ / V . Code: `ss.spearmanr`, `ss.pointbiserialr`, Cramér via `crosstab` / `statsmodels`.

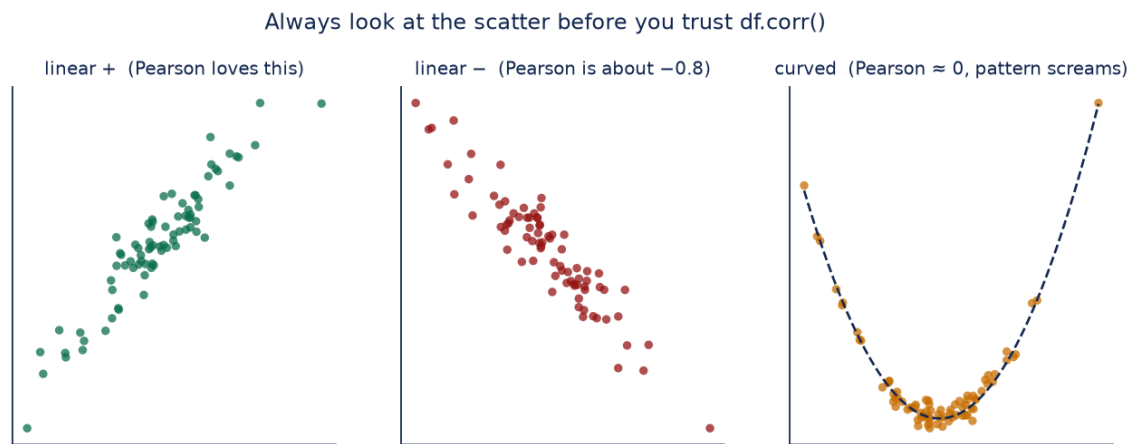


Figure — Left: Pearson’s happy place. Middle: Pearson still works (negative). Right: Pearson takes a nap while the pattern shouts.

4.4.1 Covariance, then Pearson

Derivation — step by step

Variance was “how X moves with itself”. Covariance is “how X moves with Y ”:

$$\text{cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

(Sample version; divide by n if you are in population-mode.) If x above its mean tends to meet y above its mean, the products are positive and $\text{cov} > 0$.

Covariance has ugly units (marks $\times$ years). Pearson’s r is the unitless version:

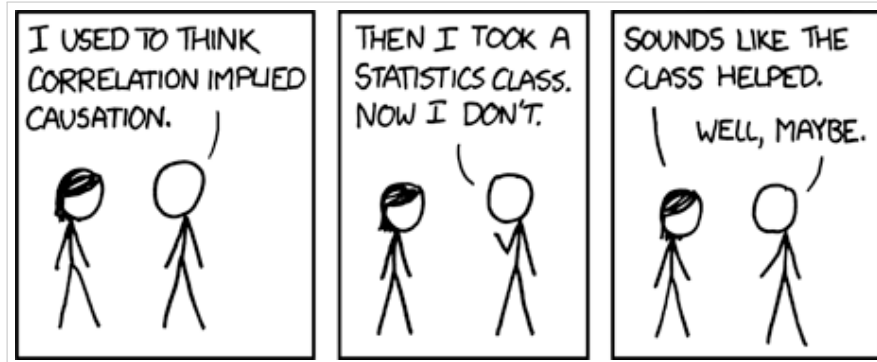
$$r = \frac{\text{cov}(X, Y)}{s_x s_y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}.$$

By Cauchy–Schwarz, $r \in [-1, 1]$. $r = 1$ is a perfect rising line; $r = -1$ a perfect falling line; $r = 0$ means *no linear* association — **not** “no association”. The board’s boomerang is the $r \approx 0$ trap.

The red ellipse on the board is the covariance contour. Its long axis is the direction of maximum variance — that arrow is principal component 1, a teaser for later courses. You do not need PCA to use the picture: fat tilted cigars mean strong $|r|$; round blobs mean $r \approx 0$.

How to write it in Python

```
1 print(df[['Age', 'IT462']].corr()) # Pearson, default
2 print(df[['Age', 'IT462']].corr(method='spearman'))
```



xkcd #552 “Correlation” by Randall Munroe — <https://xkcd.com/552/> (CC BY-NC 2.5).

The board’s most important cartoon, professionalised. Correlation is not causation; it is not even “I looked at the scatter”.

4.4.2 Spearman, point-biserial, Cramér

Derivation — step by step

Spearman r_s . Replace each x_i by its rank $R(x_i)$ (average ranks for ties), same for y , then compute Pearson on the ranks. If there are no ties there is a shortcut

$$r_s = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)}, \quad d_i = R(x_i) - R(y_i).$$

Spearman asks for a *monotone* relationship, not a straight line. That is why the board assigned it to **ordinal** columns.

Point-biserial. $X \in \{0, 1\}$, Y continuous. Let $\bar{y}_1, \bar{y}_0$ be the two group means, s_y the SD of Y on everyone, p the fraction of 1s, $q = 1 - p$:

$$r_{pb} = \frac{\bar{y}_1 - \bar{y}_0}{s_y} \sqrt{pq}.$$

It is algebraically Pearson with a 0/1 column. The board listed it separately so you do not reach for Cramér when one side is a length.

Cramér’s ϕ and V . Both start from a contingency table (crosstab) and Pearson’s χ^2 statistic

$$\chi^2 = \sum_{\text{cells}} \frac{(O - E)^2}{E}, \quad E_{rc} = \frac{(\text{row } r \text{ total})(\text{col } c \text{ total})}{n}.$$

For a 2×2 table, $\phi = \sqrt{\chi^2/n}$ (the board’s Cramér ϕ). For an $R \times C$ table,

$$V = \sqrt{\frac{\chi^2}{n \cdot \min(R - 1, C - 1)}}.$$

$V \in [0, 1]$. Nominal $\times$ nominal lives here. You cannot Pearson two city columns.

How to write it in Python

```

1 from scipy import stats as ss
2 import statsmodels.api as sm
3 from statsmodels.stats.contingency_tables import Table
4
5 r, p = ss.spearmanr(df['Age'], df['IT462'])
6 r_pb, p_pb = ss.pointbiserialr(
7     (df['Gender']=='M').astype(int), df['IT462']
8 )
9 ct = pd.crosstab(df['Gender'], df['Age']>18)
10 print(ss.contingency.association(ct, method='cramer'))
11 # equivalently: chi2, *_ = ss.chi2_contingency(ct)
12 # V = sqrt(chi2 / (ct.to_numpy().sum() * (min(ct.shape)-1)))

```

The board’s “contingency table in statsmodels” is `Table(ct)` if you want residuals and expected counts, not just V .

Watch Out

`df.corr()` will silently skip nominal columns and will Pearson anything numeric — including Likert codes you did not mean to treat as interval, and including a boomerang. **Look at the scatter.** Then pick the coefficient the board mapped.

Anscombe's quartet — identical summaries, four different stories

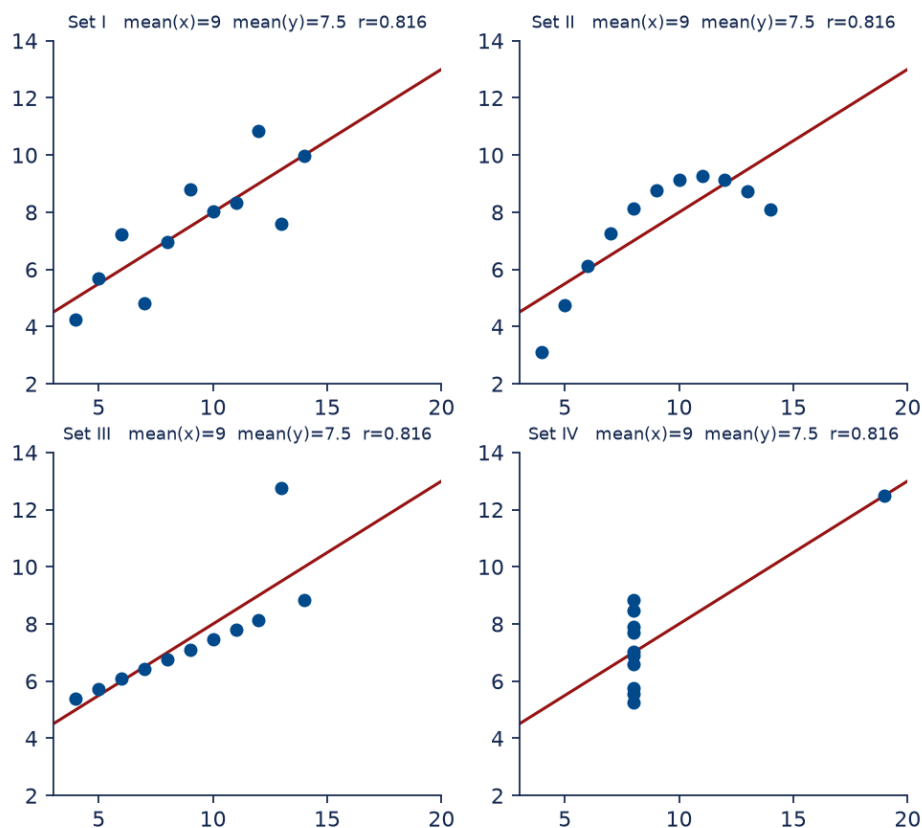


Figure — Anscombe 1973, four datasets, identical means, variances, and $r \approx 0.816$. The board’s boomerang is the same sermon with one panel.

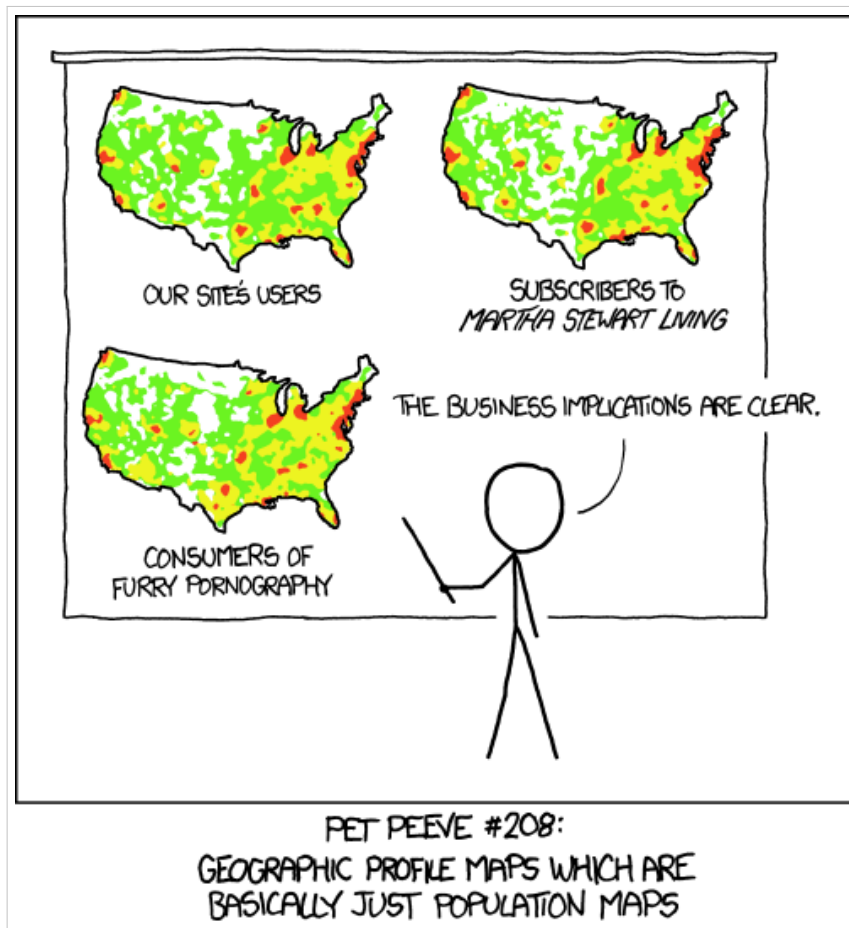
Questions & Variations

Q. Pearson of a strictly increasing log-curve?

A. $|r| < 1$ even though the relationship is perfect and monotone. Spearman will be 1. That is the point of r_s .

Q. Can r be 0 and Y still be a deterministic function of X ?

A. Yes: $Y = X^2$ on a symmetric X around 0. Compute it once in your life.



xkcd #1138 “Heatmap” by Randall Munroe — <https://xkcd.com/1138/> (CC BY-NC 2.5).

`df.corr()` as a heatmap is a poster, not a conclusion. Read the cells after you have read the scatters.

Chapter 5

Populations, Samples, and Experiments

5.1 From a column to a world

From the class board

Data $\rightarrow$ characterize (measurement scales) $\rightarrow$ statistical tools. A blob labelled **target**, overlapping “Indian population”. Experiment: effect of vaccination $\rightarrow$ **sampling**.

In Plain Words

A column is never “the world”. It is the world *as observed*. The board’s vaccination example: you care about a **target** (say, adults in India). You cannot inject and measure 1.4 billion people. You **sample**. Inferential statistics is the art of not pretending the sample *is* the target.

5.2 Parameter vs statistic

From the class board

Population oval, size N (circled) $\rightarrow$ **parameter**. Sample oval inside it, size $n \rightarrow$ each unit/entity observed $\rightarrow$ **statistic**. $N \gg n$. Question: “Who should I observe?” Combinations ${}_NC_n$.

Concept

- A **population** is the set of units you wish you could measure. Size N , possibly infinite, possibly “all future patients”.
- A **parameter** is a number that lives on the population: μ, σ, p , a regression slope. It does not jitter when you redraw a sample. We write Greek.
- A **sample** is the units you actually touch. Size n .
- A **statistic** is a number computed from the sample: $\bar{x}, s, \hat{p}, \hat{\theta}$. It jitters. We write Roman.

The whole inferential game is “statistic $\approx$ parameter, with a known kind of error”. EDA sits *before* that game and asks whether the sample is even in the right costume.

Derivation — step by step

If every subset of size n is equally likely (simple random sample, next section), the number of possible samples is the combination

$${}_NC_n = \binom{N}{n} = \frac{N!}{n!(N-n)!}.$$

The board's tiny world $N = 4$ people $\{1, 2, 3, 4\}$, $n = 2$:

$$\binom{4}{2} = 6 \quad \text{pairs: } \{12, 13, 14, 23, 24, 34\}.$$

The sketch listed 12, 13, 14 and an ellipsis — you are expected to finish the list. Each pair has probability $1/6$.

How to write it in Python

```

1 from math import comb
2 print(comb(4, 2))           # 6
3 print(comb(1_400_000_000, 1000)) # a number you should not try to
   enumerate
4 rng = np.random.default_rng(0)
5 population = np.array([1, 2, 3, 4])
6 print(rng.choice(population, size=2, replace=False))

```

Gujarat-Style Analogy

Uttarayan in Ahmedabad: the population is every kite in the sky. Your sample is the five kites you can actually keep in view from the terrace. A statistic is “3 of my 5 are yellow”. A parameter is “the fraction of yellow kites over the city”. Your terrace faces a yellow-kite shop — that is sampling bias, two sections down.

5.3 Three methods from the board (and the cousins)**From the class board**

Methods of sampling: (1) simple random, ${}_NC_n$, the $\{1, 2, 3, 4\}$ pairs; (2) cluster — population split C1..C4, whole C1 circled as the sample; (3) stratified — a rectangle split into groups, dots in each.

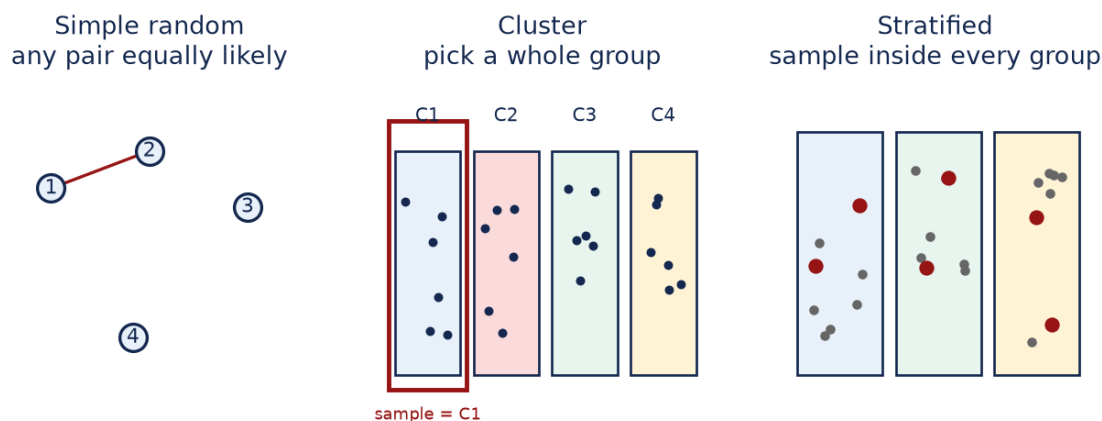


Figure — Left: any pair of the four. Middle: take a whole cluster. Right: take a little from every stratum (the red dots).

Concept

Simple random sampling (SRS). Every subset of size n has probability $1/\binom{N}{n}$. Gold standard on paper. Expensive in a country: you need a list of everyone (the sampling frame).

Cluster sampling. The population already comes in clumps (villages, schools, hospital wards, the board's C1–C4). You randomly pick *clumps* and take everyone (or a lot) inside. Cheap travel, larger sampling error: people in a clump are alike, so n bodies are not n independent stories.

Stratified sampling. You slice the population into strata you care about (state, gender, urban/rural) and SRS *inside* each slice. Guarantees that tiny but important groups appear. The board's dotted rectangles are this.

Surrounding methods the viva will ask, because they are how real data happens.

- **Systematic:** take every k th name on a list. Fine if the list is not periodic in some hidden k .
- **Convenience / voluntary:** whoever answered the Google form. Fast, biased.
- **Snowball:** ask a participant to recruit friends. Used for hidden populations; dependence is the point and the problem.

Watch Out

“Random” in English means “haphazard”. “Random” in this course means *known probabilities*. A professor grabbing the first ten students at the door is haphazard, not SRS.

How to write it in Python

```
1 df = pd.DataFrame({
2     "id": range(40),
3     "stratum": np.repeat(["urban", "rural"], 20),
4     "cluster": np.repeat(["C1", "C2", "C3", "C4"], 10),
5     "y": rng.normal(size=40),
6 })
7 srs = df.sample(n=8, random_state=0) # SRS
```

```

8 clust = df[df.cluster.eq(rng.choice(df.cluster.unique()))] # one
   whole cluster
9 strat = df.groupby("stratum", group_keys=False).apply(
10     lambda g: g.sample(n=4, random_state=0)
11 )

```

5.4 Experimental design

From the class board

Expt design. Sampling bias (arrow into problem definition). Design of experiments. Centre: Problem defn / Research question / target. Branches: experimental unit, factors / levels, response variable → choose design.

Concept

Write these down *before* collecting, or you will collect a puddle.

- **Research question.** A sentence with a verb: “does vaccine *A* reduce hospitalisation in adults over 60?”
- **Target.** To whom will you generalise?
- **Experimental unit.** The smallest thing that independently receives a treatment: a person, a plot of land, a classroom (not a student, if the whole class got the same lesson).
- **Factors and levels.** The knobs you turn (drug: placebo / low / high) and the values they take.
- **Response.** What you measure (fever at day 7, marks, yield).
- **Sampling bias.** The terrace facing the yellow-kite shop. If your vaccination sample is “whoever walked into this one private clinic”, your parameter is no longer “India”.

From the class board

Three headings above Collect data: **Randomization, Replication, Control / Block.** Then IDA — “Is the data reliable?” Then EDA — “What patterns are there?”

Derivation — step by step

Fisher’s three Rs, in one paragraph each.

Randomization. Assign treatments with a known chance process. This does two jobs: it balances lurking variables *on average* (the health-conscious people do not all pick the vaccine arm), and it licenses the probability model behind a *p*-value. Without it you have an observational study — still useful, harder to causal-ise.

Replication. Repeat the experiment on more than one unit. $n = 1$ is an anecdote. Replication also means: can a stranger rerun your notebook on a new draw and land near your number?

Control / blocking. **Control** is a comparison arm (placebo, old drug, “no interven-

tion”). **Blocking** is stratified assignment: randomize *inside* gender or hospital so those factors do not swamp the treatment. A randomised complete block design is “stratified sampling” for experiments.

Gujarat-Style Analogy

A kite-fight: randomization is tossing a coin for who gets the new manja; replication is more than one fight; control is a neighbour still using old manja; blocking is not mixing the 6-year-olds and the 20-year-olds into one “average fight”.

How to write it in Python

```
1 units = pd.DataFrame({"id": range(20), "block": np.repeat(["M","F"],
2                       10)})
3 units["arm"] = (
4     units.groupby("block")["id"]
5     .transform(lambda s: rng.permutation(["vaccine","placebo"]
6     *(len(s)//2)))
7 )
8 print(units.head())
```

Questions & Variations

Q. Cluster vs stratified, in one sentence?

A. Cluster *samples clumps*. Stratified *forces every clump-type to appear*. You cluster for logistics. You stratify for representation.

Q. Is a census a sample?

A. If you truly got everyone, $n = N$ and the statistic *is* the parameter. In practice censuses miss people; treat them as very large, biased samples.

Q. Where does IDA sit relative to randomization?

A. After collection, before you brag about patterns. Randomization does not stop a column from being coded -99 for missing.

```
int getRandomNumber()
{
    return 4; // chosen by fair dice roll.
             // guaranteed to be random.
}
```

xkcd #221 “Random Number” by Randall Munroe — <https://xkcd.com/221/> (CC BY-NC 2.5).

If your sampling process is “I thought about it and picked 4”, you have invented a new kind of bias, not a random number.

Chapter 6

After Import: The Practitioner's Loop

This chapter is Lecture 10 plus the surrounding craft that makes a 2026 analyst dangerous in a good way.

6.1 The feature checklist

From the class board

Data Import (.csv). 1. **features** vs **target feature**. Then: (1) dimension `df.shape`; (2) type (measurement scales); (3) naming convention; (4) units, date-time (values); (5) duplication; (7) feature engineering (add-on, double-ticked).

In Plain Words

After `read_csv` you do not fit a model. You walk a checklist. The board split the walk in two: questions about **columns** (this section) and questions about **cells** (next). The **target** is the column you would like to predict or explain; the rest are features. Even in pure EDA, naming the target stops you from “exploring” in a circle.

How to write it in Python

```
1 df = pd.read_csv("data.csv")           # or pl.read_csv, or duckdb.  
    read_csv  
2 print(df.shape)                        # (1) n observations, p  
    features  
3 print(df.dtypes)                       # (2) machine types --- then *  
    you* assign NOIR  
4 print(df.columns.tolist())             # (3) names  
5 df.columns = (df.columns  
6               .str.strip().str.lower()  
7               .str.replace(r"[^a-z0-9]+", "_", regex=True))  
8 # (4) units live in a codebook; datetimes must be parsed, not left  
    as strings  
9 df["when"] = pd.to_datetime(df["when"], errors="coerce", utc=True)  
10 print(df.duplicated().sum())           # (5)
```

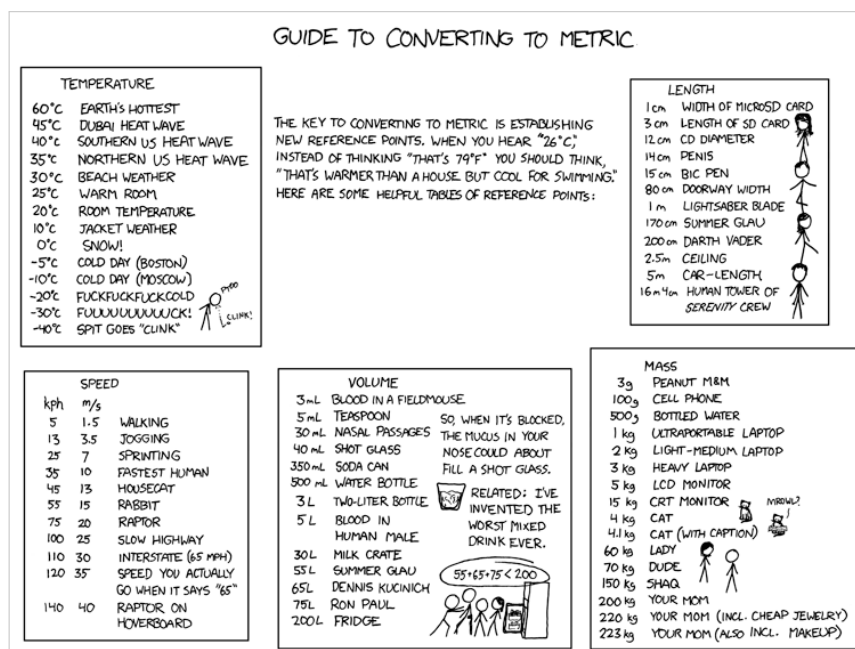
Concept

Naming. Spaces, mixed case, and Age (yrs) will rot every later line of code. Snake-case, ASCII, units in the codebook not the name, is a 2026 hygiene that pandas does not enforce and Pandera can.

Units and datetimes. “90” is meaningless. 90 kg, 90 min, 90 mmHg are three worlds. Dates arriving as 01/02/03 are a war crime; parse with an explicit format. Store UTC, display local.

Duplicates. Entire-row copies are usually a join accident. Duplicate IDs with different payload are a real-world collision — the dirty table from Chapter 1. Decide: keep last, keep first, or explode into a conflict table. Do not `drop_duplicates()` as a reflex.

Feature engineering (the board’s add-on). New columns made from old ones: age-from-DOB, log-income, one-hot city, “is_missing” flags, target encoding with extreme caution (leakage). Engineering is CDA-adjacent. Do it after you have looked, and do it on train folds if you are confirming.



xkcd #526 “Converting to Metric” by Randall Munroe — <https://xkcd.com/526/> (CC BY-NC 2.5).

Units are a measurement-scale problem in disguise. Mixing cm and inches is how you invent outliers.

6.2 The value checklist

From the class board

2. values / observations: range (feasible) — min, max; unitless; duplicates; missing; outlier.

How to write it in Python

```
1 print(df.describe(include="all").T[["min", "max"] if False else df.
   columns[:0]])
2 num = df.select_dtypes("number")
3 print(num.agg(["min", "max"]))
4 # feasible range: age in [0, 120], marks in [0, 100], gender in {M,F
```

```

, None}
5 bad_age = ~df["age"].between(0, 120) & df["age"].notna()
6 print(df.isna().mean().sort_values(ascending=False))    # missingness
rate

```

Concept

Feasible range. The board underlined “feasible”. Min and max are not just summaries; they are IDA alarms. Age –1, marks 110, latitude 1000.

Unitless. Some derived features (proportions, z -scores, CVs, r) have no unit. Write that down so nobody “converts” them.

Missing. Three classical mechanisms (Little & Rubin):

- **MCAR** — missingness is a coin toss, independent of data. Rare.
- **MAR** — missingness depends on *observed* columns (men skip the “how many children” question). Imputation using those columns is defensible.
- **MNAR** — missingness depends on the missing value itself (the richest people leave income blank). You cannot fix this with `fillna` and a smile.

Plot missingness (`df.isna()` heatmap, or the `missingno` library). A column that is 60% empty is a feature-engineering decision, not a mean-impute.

Outliers. A point that is surprising *given a model of “typical”*. The model can be “beyond 1.5 IQR from the box”, or “ $|z| > 3$ ”, or “a doctor looked at it”. Boxplots (Chapter 3 figure) make the 1.5 IQR rule visible. Never delete outliers as a ritual: yesterday’s outlier is tomorrow’s signal (fraud, a pandemic, a 99-mark genius).

Derivation — step by step

Tukey’s boxplot fences, since the board said “outlier” and every library uses them:

$$\text{lower fence} = Q_1 - 1.5 \text{ IQR}, \quad \text{upper fence} = Q_3 + 1.5 \text{ IQR}.$$

A point outside is a *candidate*. The 1.5 is a convention (Tukey thought it caught about the last 0.7% of a Gaussian). For a z -score on ratio/interval data that is not wildly skewed,

$$z_i = \frac{x_i - \bar{x}}{s}.$$

$|z_i| > 3$ is another candidate flag. On right-skewed income, log first or use MAD:

$$z_i^{\text{MAD}} = \frac{x_i - \tilde{x}}{1.4826 \text{ MAD}}.$$

6.3 Encoding: how NOIR enters a model

In Plain Words

Computers want numbers. NOIR tells you *which* numbers are a translation and which are a lie.

How to write it in Python

```

1 # nominal -> one-hot (no fake order)
2 pd.get_dummies(df["gender"], dummy_na=True, prefix="g")
3
4 # ordinal -> ordered codes (fake equal gaps, on purpose)
5 size = pd.Categorical(df["size"], categories=["S","M","L","XL"],
6                       ordered=True)
7 df["size_code"] = size.codes
8
9 # binary -> 0/1, then point-biserial is waiting
9 df["is_urban"] = (df["stratum"]=="urban").astype(int)
10
11 # never: LabelEncoder on city names for a linear model

```

Watch Out

Target encoding (replace a city by its mean marks) leaks the target into a feature. If you confirm a model, compute the means on the training fold only. This is the 2018–2026 Kaggle footgun, still loaded.

6.4 A visual grammar (so your plots are not souvenirs)

Concept

Wickham’s layered grammar (2009), which ggplot2 / plotnine / Vega-Lite / Altair all speak: a plot is

data + mapping (aes) + geom + stat + scale + facet.

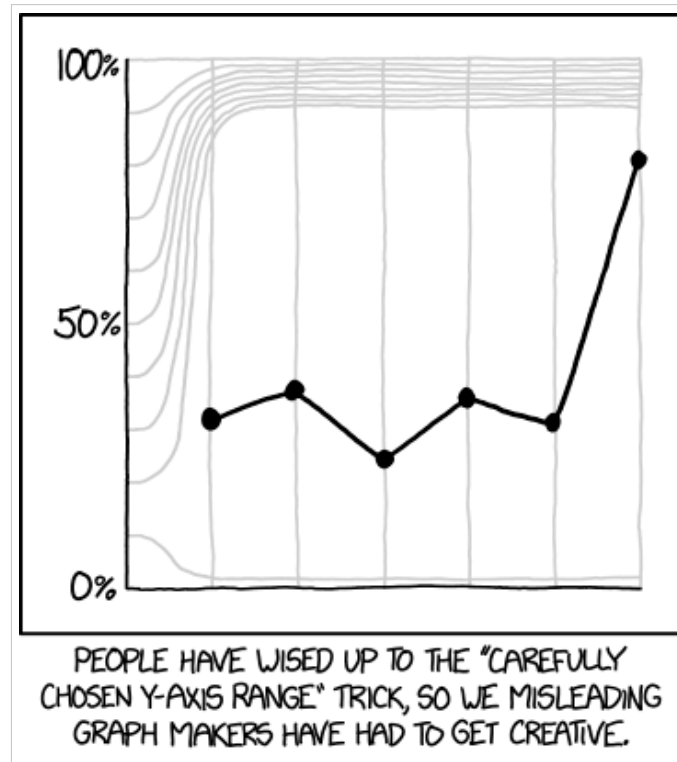
EDA in 2026 is not “I called `sns.heatmap`”. It is: I mapped this ratio-column to *y*, this ordinal to colour, this nominal to facet, and I chose a geom that respects NOIR (bar for nominal, hist/box for ratio, scatter for two ratios). The board never wrote “grammar of graphics”. Every good plot you have ever seen was speaking it.

How to write it in Python

```

1 import seaborn as sns
2 sns.histplot(df, x="age") # one ratio
3 sns.boxplot(df, x="gender", y="IT462") # nominal vs ratio
4 sns.scatterplot(df, x="age", y="IT462", hue="gender")
5 sns.heatmap(df.select_dtypes("number").corr(), annot=True, vmin=-1,
6             vmax=1)

```



xkcd #2023 "Y-Axis" by Randall Munroe — <https://xkcd.com/2023/> (CC BY-NC 2.5).

A dishonest axis is CDA in a clown suit. Start at zero for counts and bars; do not start at zero for temperature anomalies just because a blog said so — know *why*.

6.5 The 2026 stack, without throwing away the board

2026 cutting edge

What non-generic threads on r/datascience and r/MachineLearning actually push as of August 2026, and how they sit on IT462:

Polars — a Rust DataFrame, parallel, lazy. Same NOIR, faster `read_csv` on the file that made pandas swap. Expressions replace row-wise `apply`.

DuckDB — SQLite's analytical cousin. SQL over Parquet / CSV / a Polars frame, in-process, no warehouse. Use it when the question is already SQL ("median marks by city having $n > 30$ ").

Parquet + Arrow — columnar files as the source of truth. CSV is a letter you send; Parquet is the filing cabinet.

Pandera / Great Expectations — IDA as code: "age is ratio in $[0, 120]$, gender is nominal in $\{M, F\}$, ID is unique". The board's "usable: valid, reliable, authentic, error-free" becomes a failing CI job.

ydata-profiling — one call, a HTML portrait of every column. A start, not a substitute for looking at the boomerang.

LLM-assisted EDA (LIDA 2023 → DA-Studio 2026, DataClawBench, Walk-Before-You-Run, Aug 2026). Agents that *write* the pandas/Polars you should then read. The papers of 2026 are all, in different accents, Tukey's warning: exploration is not a vibe, it is an evaluable stage. If the agent cannot show the code, it is not EDA, it is astrology.

How to write it in Python

```

1 # Polars, the 2026 first look
2 import polars as pl
3 lf = pl.scan_csv("data.csv")
4 print(lf.collect_schema())
5 print(lf.select(pl.all().is_null().mean()).collect())
6
7 # DuckDB, the SQL first look
8 import duckdb
9 print(duckdb.sql("""
10     SELECT gender, median(IT462), count(*) AS n
11     FROM 'data.csv'
12     GROUP BY 1
13     ORDER BY n DESC
14 """).pl())

```



xkcd #1838 “Machine Learning” by Randall Munroe — <https://xkcd.com/1838/> (CC BY-NC 2.5).

CDA without this chapter is how you get a very confident, very wrong model. EDA is the adult in the room.

6.6 A full walk on the dirty table

How to write it in Python

```

1 from scipy import stats as ss
2 df = pd.DataFrame({
3     "id":      ["2026.", "2026", None, None, "2023"],
4     "IT462":   [10, 50, 90, -20, 17],
5     "age":     [0, 10, 20, 100, np.nan],
6     "gender":  ["M", "F", "M", "F", np.nan],
7 })
8 df["id"] = df["id"].astype("string").str.replace(r"\.$", "", regex=
    True)
9 df["gender"] = pd.Categorical(df["gender"], categories=["M","F"],
    ordered=False)
10 # IDA
11 print("shape", df.shape)
12 print(df.dtypes)
13 print("dup ids", df["id"].duplicated(keep=False))
14 print("infeasible", df.query("IT462 < 0 or age > 90"))
15 print("missing %\n", df.isna().mean())
16 # EDA --- legal summaries only
17 print("gender mode", df["gender"].mode().tolist())          #
    nominal
18 print("age median/mean", df["age"].median(), df["age"].mean()) #
    ratio
19 print("IT462 skew", df["IT462"].skew())
20 print(ss.spearmanr(df["age"], df["IT462"], nan_policy="omit"))

```

Every function here was a box on a lecture board. That is the course.

Questions & Variations

Q. First line of code after `read_csv`?

A. `df.shape`, then `dtypes` / a NOIR table you wrote by hand, then missingness, then describe, then plots. Not `LinearRegression().fit`.

Q. Why might `.skew()` on 5 rows be a clown number?

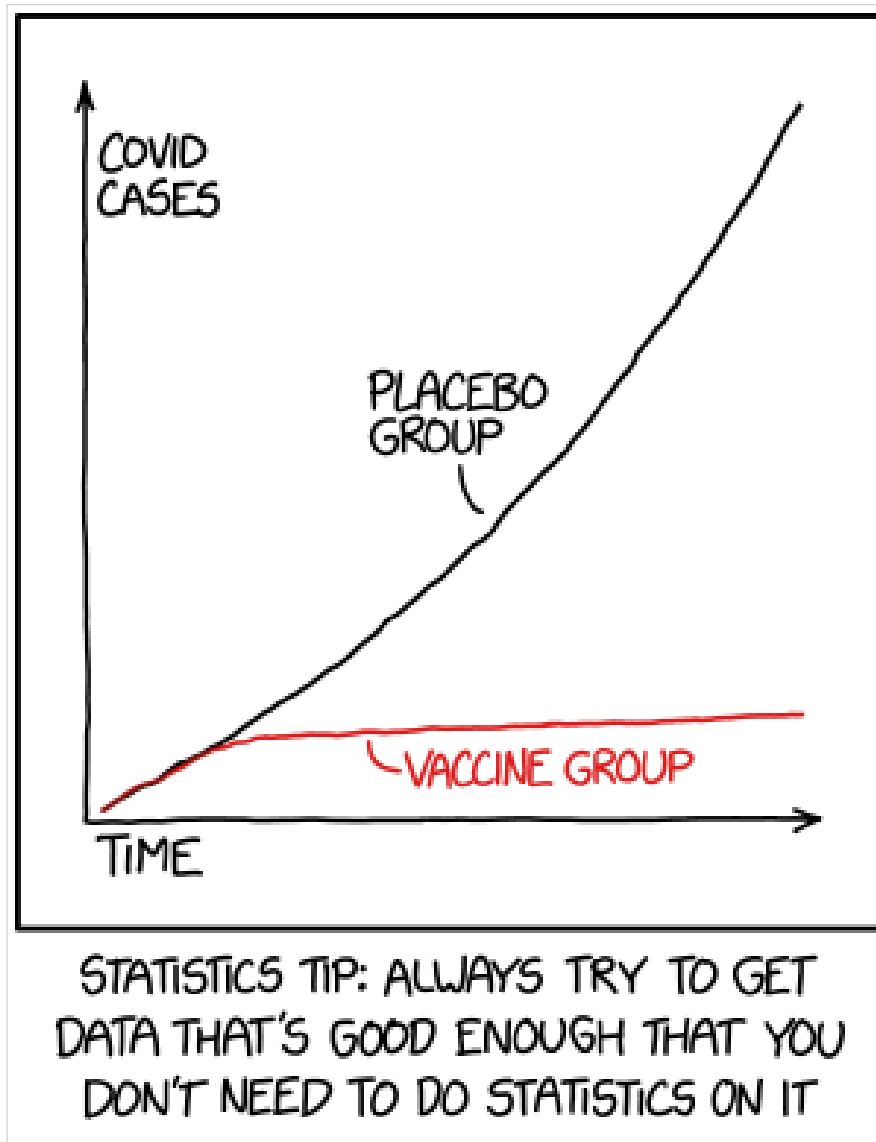
A. Moment skewness is a tail story. $n = 5$ has no tails, only anecdotes. Quote the cartoon (mean vs median) instead.

Q. Name three 2026 tools that implement IDA as code.

A. Pandera, Great Expectations, (and a well-configured ydata-profile with constraints). LLM agents only count if they emit the constraint code.

Q. Pearson or Spearman for Likert vs marks?

A. Spearman, unless you have argued the Likert-as-interval transform and shown the scatter is not a boomerang.



xkcd #2400 "Statistics" by Randall Munroe — <https://xkcd.com/2400/> (CC BY-NC 2.5).

You are now the person in the third panel. Go and look at some real tables.

Closing: The Whole Story in One Page

Raw stuff is **data**. A number you can act on is **information**. IDA asks “is this usable?” EDA asks “what is the shape of this usable thing?” CDA asks “does my model survive a fair test?”

Columns are features. Rows are observations. Every feature has a **measurement scale** (NOIR). The scale decides which operations, which plots, and which association coefficient are legal. Central tendency is a pin in the map; dispersion is how spread the pins are; skewness and kurtosis are the *shape* of the pile; association is how two piles move together. A sample statistic is a noisy photograph of a population parameter. Randomization, replication, and control are how you stop the photograph from being a selfie of your own bias.

After `pd.read_csv`, you do not model. You check shape, types, names, units, time, duplicates, missingness, feasible ranges, and outliers. Then you look. Then — and only then — you encode, engineer, and confirm.

Annotated bibliography (what we actually opened)

All of the following were downloaded into `sources/` (or already sat in the FOML library) and *read for ideas*. Nothing is copy-pasted.

1. Allen B. Downey, *Think Stats* 2e (Green Tea Press, CC). Still the least generic “EDA in Python” book; 3e (2024–26) moved to notebooks, 2e remains the PDF we used.
2. Hadley Wickham, “Tidy Data”, *J. Statistical Software* 59 (2014).
3. Hadley Wickham, “A Layered Grammar of Graphics”, *Journal of Computational and Graphical Statistics* (2009). The reason `seaborn` / `Altair` / `plotnine` feel the same in 2026.
4. F. J. Anscombe, “Graphs in Statistical Analysis”, *The American Statistician* (1973).
5. Victor Dibia, *LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models* (arXiv:2303.02927). Ancestor of 2026 agentic EDA.
6. DA-Studio (arXiv:2606.31423, 2026). End-to-end inspectable agentic analysis — the 2026 version of “don’t trust the chatbot, watch the trace”.
7. “Business Utility of Large Language Models as Exploratory Data Analysis Agents” (arXiv:2606.00051, 2026). Reliability / repeatability of EDA agents.
8. DataClawBench (arXiv:2605.02503, 2026). Benchmark for agents on noisy, underexplored tables.
9. “Walk Before You Run” (arXiv:2608.16045, Aug 2026). Makes data exploration a first-class, evaluable stage in agent pipelines — the anti-hype paper of the month.
10. Polars user guide, Lazy API (docs.pola.rs, accessed Aug 2026). The DataFrame people actually recommend on Reddit in 2026 when pandas starts swapping.
11. ydata-profiling / pandas-profiling literature (arXiv:1912.03511 and follow-ups).
12. DuckDB literature (arXiv:2208.07127 and the in-process OLAP line). Local-first EDA on Parquet without standing up a warehouse.
13. Mathematics for Machine Learning (Deisenroth, Faisal, Ong) — free PDF, used for the covariance-geometry picture.
14. CS229 notes (Stanford), supervised-learning chapter — how EDA sits under a model.
15. Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* — least squares as the CDA that EDA is allowed to dream about, not skip to.
16. Gareth James et al., *An Introduction to Statistical Learning* (ISLR PDF). The 2026 Reddit staple for “what happens after EDA”. Not a substitute for Tukey.

Reddit / internet pulse, August 2026 (rewritten, not scraped as images): the non-generic stack is no longer “pandas + seaborn and a smile”. It is **Parquet as the source of truth, Polars or DuckDB for the first look, Pandera / Great Expectations as the IDA gate**, and **LLM agents that emit audited code** rather than vibes. Tukey still wins the philosophy argument. Wickham still wins the table argument. Anscombe still wins the “look at the picture” argument.